



Learning-based query optimization for multi-probe approximate nearest neighbor search

Pengcheng Zhang¹ · Bin Yao^{1,3} · Chao Gao¹ · Bin Wu² · Xiao He² · Feifei Li² · Yuanfei Lu² · Chaoqun Zhan² · Feilong Tang¹

Received: 23 May 2021 / Revised: 22 June 2022 / Accepted: 22 August 2022 / Published online: 12 September 2022
© The Author(s), under exclusive licence to Springer-Verlag GmbH Germany, part of Springer Nature 2022

Abstract

Approximate nearest neighbor search (ANNS) is a fundamental problem that has attracted widespread attention for decades. Multi-probe ANNS is one of the most important classes of ANNS methods, playing crucial roles in disk-based, GPU-based, and distributed scenarios. The state-of-the-art multi-probe ANNS approaches typically perform in a fixed-configuration manner. For example, each query is dispatched to a fixed number of partitions to run ANNS algorithms locally, and the results will be merged to obtain the final result set. Our observation shows that such fixed configurations typically lead to a non-optimal accuracy–efficiency trade-off. To further optimize multi-probe ANNS, we propose to generate efficient configurations for each query individually. By formalizing the per-query optimization as a 0–1 knapsack problem and its variants, we identify that the k NN distribution (the proportion of k nearest neighbors of a query placed in each partition) is essential to the optimization. Then we develop LEQAT (LEarned Query-Aware OpTimizer), which leverages k NN distribution to seek optimal configurations for each query. LEQAT comes with (i) a machine learning model to learn and estimate k NN distributions based on historical or sample queries and (ii) efficient query optimization algorithms to determine the partitions to probe and the number of searching neighbors in each partition. We apply LEQAT to three state-of-the-art ANNS methods IVF, HNSW, and SSG under clustering-based partitioning, evaluating the overall performance on several real-world datasets. The results show that LEQAT consistently reduces the latency by up to 58% and improves the throughput by up to 3.9 times.

Keywords Approximate nearest neighbor search · Query optimization · Machine learning

1 Introduction

Nearest neighbors search (NNS) plays a fundamental role in various applications, e.g., database system [9,54,56,57], information retrieval [44,46], data mining [1,7], and machine learning [20,58]. As vector dimensionality and database size grow, exact NNS algorithms lead to unprecedented computation cost. The experimental results have demonstrated that the exact methods perform no better than the exhaustive search when the dimensionality is high [52]. Thus, approximate nearest neighbor search (ANNS) [25] algorithms are widely leveraged to address this problem. ANNS methods provide a trade-off between search accuracy and efficiency by returning a subset of the ground-truth k NNs.

✉ Bin Yao
yaobin@cs.sjtu.edu.cn
Pengcheng Zhang
zhangpc@sjtu.edu.cn
Chao Gao
dimon-gao@sjtu.edu.cn
Bin Wu
binwu.wb@alibaba-inc.com
Xiao He
xiao.hx@alibaba-inc.com
Feifei Li
lifeifei@alibaba-inc.com
Yuanfei Lu
hanming.lyf@alibaba-inc.com
Chaoqun Zhan
lizhe.zcq@alibaba-inc.com
Feilong Tang
tang-fl@cs.sjtu.edu.cn

¹ Shanghai Jiao Tong University, Shanghai, China

² Alibaba Cloud, Hangzhou, China

³ Hangzhou Institute of Advance Technology, Hangzhou, China

Multi-probe ANNS methods [15,29,53,55] are characterized as one class of the most important ANNS approaches. They employ partitioning techniques (e.g., clustering) to split a dataset into multiple partitions. ANNS is performed in a multi-probe manner, i.e., probing multiple partitions to search and gather k NNs. Multi-probe ANNS methods have many important applications. Firstly, they are widely used in disk-based databases/systems [29,55], where each partition is saved as an individual file or disk block. These systems play important roles in face recognition [29] or similar product search in e-commerce scenarios [55]. Secondly, the partitioning technique can be combined with vector quantization to support high-throughput ANNS on GPUs [29]. At last, several ANNS solutions or systems [10,53] leverage multi-probe ANNS to process large datasets on a cluster, as different partitions can be naturally stored on different machines [10,15,53]. In particular, Microsoft SPTAG [10] is successfully used in the Bing search engine for serving online similarity search.

In this paper, we focus on the following two important categories of multi-probe ANNS approaches:

1. *One-level approaches* (or flat approaches) that store the original vectors in each partition. ANNS is performed by first choosing top- M partitions according to specific heuristics and searching the selected partitions exhaustively. For example, the IVF-Flat index in Faiss [29] and PASE [55] split a dataset into multiple partitions through K-means clustering and performs ANNS by probing top- M partitions whose clustering centroids are closest to the query vector.
2. *Two-level approaches* (graph-based distributed approaches) [15,53] that divide the dataset into multiple partitions, build a graph index for each partition, and perform ANNS through the graph indexes. For example, the work [15] splits the dataset into multiple partitions randomly, building an NSG index for each partition to serve distributed ANNS. ADBV [53] partitions a dataset through K-means clustering and performs top- k ANNS by: (i) choosing M partitions whose cluster centroids are closest to the query vector; (ii) searching c candidate nearest neighbors in each graph index of chosen partitions in parallel; (iii) gathering results from local partitions and merging them into the final result set.

In general, the performance of multi-probe ANNS is determined by: (i) The partition pruning strategy (for both one-level and two-level approaches). A better partition pruning strategy leads to fewer partitions to be probed. (ii) The number of candidate neighbors searched in each partition (for only two-level approaches). In order to find k NNs and ensure search accuracy, SOTA graph-based ANNS approaches return top- c candidate neighbors, where c is

larger than k . A larger k needs a larger c to ensure query accuracy, and a larger c leads to larger latency. Thus the value of c for each partition influences the search performance significantly.

We use the following example to demonstrate the effect of the partition pruning strategy and searching neighbors in each partition. Suppose that one dataset is divided into 512 partitions, and graph indexes are built for them. A query's 10NNs are placed in two partitions where partition-1 has eight of the 10NNs, and partition-2 has two of the 10NNs, respectively. Under such a situation, the most efficient way to find the 10NNs is to search only those two partitions by setting a larger c in partition-1 and a smaller c in partition-2. However, since the information on how many k NNs are placed in each partition is unknown before the query execution, the most efficient way to execute a query is not apparent. As a result, existing approaches typically leverage fixed configurations to balance search accuracy and efficiency, such as searching a fixed number of partitions and searching a fixed number of candidate neighbors in each partition.

Our study finds that using fixed configurations ignores the distinctiveness of queries and partitions, thus leading to a non-optimal accuracy–efficiency trade-off. To further optimize multi-probe ANNS, we present the LEQAT optimizer, a novel learning-based approach that determines query-aware configurations for each query individually. By formalizing the query-aware configuration for one-level and two-level approaches as two optimization problems, we demonstrate that the k NN distribution (the proportion of k nearest neighbors of a query placed in each partition) plays a critical role in optimization. We then develop several models to learn and estimate the underlying k NN distribution of each query. Based on k NN distribution estimation, we then introduce algorithms to efficiently determine query-aware configurations for each query. Firstly, we propose two partition pruning methods to determine the partitions to be probed. Secondly, we introduce an enlarging factor δ to calibrate the c parameter for each query and each partition for two-level approaches. We ensure c is proportional to the estimated number of ground-truth k NNs placed in each partition. We apply LEQAT to three state-of-the-art ANNS indexes IVF [29], HNSW [38], and SSG [14] under clustering-based partitioning to evaluate the end-to-end performance. LEQAT consistently reduces the latency by up to 58% and improves the throughput by up to 3.9 times compared to fixed configurations under the same accuracy targets.

Our major contributions could be summarized as follows:

1. We identify inefficiencies of the state-of-the-art multi-probe ANNS approaches due to using fixed configurations.

2. We illustrate that the k NN distribution is essential for optimizing multi-probe ANNS by formalizing the query optimization problems.
3. We propose machine-learning models to learn and estimate queries' k NN distributions.
4. We present novel query-aware configuration techniques that efficiently determine configurations for each query individually.
5. We conduct extensive experiments to demonstrate the effectiveness and efficiency of our proposed approach.

The rest of this paper is organized as follows. Section 2 introduces the preliminary knowledge. Section 3 presents our exploratory studies on real-world datasets to illustrate the motivation. Section 4 formulates the multi-probe ANNS optimization problems. Section 5 details our approaches to generate query-aware configurations and tune hyperparameters. Section 6 provides the experimental results. The related work and conclusion are presented in Sects. 7 and 8.

2 Preliminary

2.1 ANNS overview

The k nearest neighbors (k NN) search problem aims to find the k nearest neighbors in a dataset to a given query vector [45]. Formally, the problem is defined as:

Definition 1 *k Nearest Neighbors (kNN) Search Problem* Let $\mathcal{D} = \{x_1, x_2, \dots, x_n\}$ be a dataset consisting of n vector points in a d -dimensional space $\mathbb{R}^d$, and $\text{dist}(x_i, x_j)$ denote a pair-wise distance function between two vectors x_i and x_j . Given a query vector $q \in \mathbb{R}^d$ and an integer $k \geq 1$, k nearest neighbors search returns q 's k nearest neighbors set $\text{TopK}_q \subseteq \mathcal{D}$, where

$$\forall v \in \text{TopK}_q, \forall x \in \mathcal{D} \setminus \text{TopK}_q, \text{dist}(v, q) \leq \text{dist}(x, q).$$

ANNS returns a set TopK'_q containing a proportion (or even none) of ground-truth answers in TopK_q . Recall is used to measure the accuracy of ANNS algorithms. For a given query q , the recall is defined as:

$$\text{Recall} = \frac{|\text{TopK}'_q \cap \text{TopK}_q|}{k} \quad (1)$$

In this paper we focus on ANNS under the Euclidean distance and angular distance, due to their wide use in various domains [1,7,9,20,44,46,54,56,57].

Given two d -dimensional vectors $\mathbf{x}$ and $\mathbf{y}$, the Euclidean l_2 distance is defined as:

$$\text{EucL}_2(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^d (x_i - y_i)^2 \quad (2)$$

The angular distance, also known as cosine dissimilarity, measures the angle between two vectors:

$$\text{AngD}(\mathbf{x}, \mathbf{y}) = 1 - \frac{\sum_{i=1}^d x_i y_i}{\sqrt{\sum_{i=1}^d x_i^2} \sqrt{\sum_{i=1}^d y_i^2}} \quad (3)$$

Note that the angular distance can be expressed via the Euclidean l_2 distance under the condition that vectors are normalized to unit vectors, i.e., under the condition that $\sqrt{\sum_{i=1}^d x_i^2} = 1$ and $\sqrt{\sum_{i=1}^d y_i^2} = 1$:

$$\begin{aligned} \text{AngD}(\mathbf{x}, \mathbf{y}) &= 1 - \frac{\sum_{i=1}^d x_i y_i}{\sqrt{\sum_{i=1}^d x_i^2} \sqrt{\sum_{i=1}^d y_i^2}} \\ &= 1 - \sum_{i=1}^d x_i y_i \\ &= 1 - \frac{\sum_{i=1}^d (x_i^2 + y_i^2)}{2} + \frac{\sum_{i=1}^d (x_i^2 + y_i^2 - 2x_i y_i)}{2} \\ &= 1 - \frac{\sum_{i=1}^d x_i^2 + \sum_{i=1}^d y_i^2}{2} + \frac{\sum_{i=1}^d (x_i - y_i)^2}{2} \\ &= \frac{\sum_{i=1}^d (x_i - y_i)^2}{2} \\ &= \frac{\text{EucL}_2(\mathbf{x}, \mathbf{y})}{2} \end{aligned} \quad (4)$$

2.2 Overview of multi-probe ANNS approaches

As described in the introduction, multi-probe ANNS can benefit many real-world applications, especially in disk-based, GPU-based, and distributed settings. Multi-probe approaches are built upon the data partitioning technique, i.e., the dataset must first be split through certain partitioning methods (e.g., K-means clustering). Afterward, the execution of ANNS methods could be decomposed into three steps:

1. *Partition pruning* Select M closest partitions of the query vector as candidates for the next step;
2. *Local search* Perform k NN algorithms at each of the local candidate partitions;
3. *Gathering* Gather local results returned from M partitions to build the final result set.

According to the indexing/searching method involved in the second step, multi-probe approaches could be further

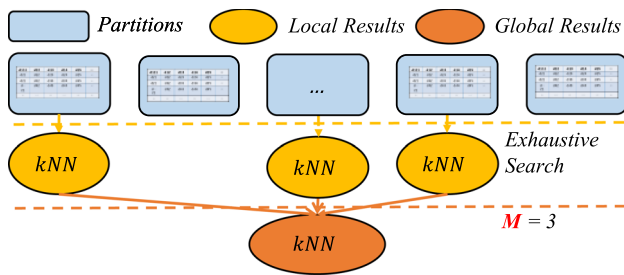


Fig. 1 Overview of one-level multi-probe ANNS

divided into two categories: one-level approaches and two-level approaches.

One-level multi-probe ANNS approaches One-level approaches have no more indexing phases for local partitions, and they store the database vectors directly in each partition. The IVF-Flat (IVF means Inverted File) approach in Faiss [29] and the multi-probe LSH approach [36] (under the condition that using one hash table) both fall into this category.

As shown in Fig. 1, one-level approaches follow the three ANNS steps as described previously, where the second local searching step is performed through an exhaustive search. So, the performance of one-level approaches is majorly determined by the M parameter. A larger M value results in more partitions to be probed and more vectors to be visited, increasing search accuracy but decreasing search efficiency.

Two-level multi-probe ANNS approaches Recently proposed graph-based ANNS indexes have demonstrated excellent search performance, thus are widely used in low-latency scenarios [15,53]. However, the high memory requirement of graph-based ANNS indexes makes them hard to be applied to large datasets on a single machine. Therefore, data partitioning techniques are leveraged to split a dataset into multiple partitions. Separate graph-based indexes are built for individual partitions, and each of them could fit in a single machine's memory. These indexes will be distributed to multiple machines for distributed ANNS. Such an approach is used in several large-scale ANNS solutions or systems [15,29,53] for distributed ANNS and have shown great scalability [53].

As shown in Fig. 2, the only difference between two-level approaches and one-level approaches is the local searching step. Two-level approaches perform local searching through the graph indexes instead of exhaustively searching local partitions. For top- k ANNS, graph-based approaches generally return top- c ($c \geq k$) candidate neighbors. Here, the larger value of c is used to collect more candidate neighbors in the graph traversing procedure. Then, top- k nearest vectors are obtained by sorting these candidate neighbors. So, the performance of two-level multi-probe ANNS approaches is significantly influenced by the M and c parameters. Larger

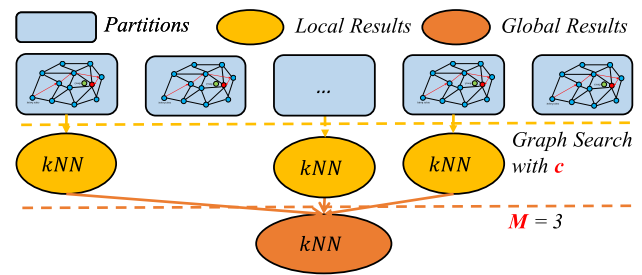


Fig. 2 Overview of two-level multi-probe ANNS

M and c values result in more partitions to be probed and more candidates to be searched, increasing search accuracy but decreasing search efficiency.

2.3 Knapsack problem

0–1 Knapsack Problem Given the weights and profits of N items, the 0–1 knapsack problem [13] is to put these items into a knapsack with limited capacity W , to maximize the profit from items in the knapsack, under the constraint that each item can only be selected once. Let x_i be an indicator variable that determines whether the i th item will be selected, and w_i and v_i are the weight and profit of the i th item, the 0–1 knapsack problem can be formulated as

$$\max_{x_i \in \{0,1\}} \sum_{i=1}^N x_i v_i \quad (5)$$

$$\text{s.t.} \quad \sum_{i=1}^N x_i w_i \leq W \quad (6)$$

There is an $O(NW)$ algorithm that solves this problem exactly using dynamic programming [13].

Multiple-Choice Knapsack Problem (MCKP) If the N items are divided to n groups $S_1, \dots, S_n$, where each group S_i has m_i items, and we require to choose exactly one item from each group such that the total profit is maximized without exceeding the capacity W . This problem is known as the multiple-choice knapsack problem (MCKP). Let x_{ij} be an indicator variable determining whether the j th item in the i th group will be selected, and the constraint that exact one item is selected in a group can be expressed by $\sum_{j=1}^{m_i} x_{ij} = 1, \forall i \in \{1, \dots, n\}$. Then, let v_{ij} and w_{ij} denote the profit and weight of the j th item in the i th group, respectively, the MCKP problem is formulated as

$$\max_{x_{im} \in \{0,1\}} \sum_{i=1}^n \sum_{j=1}^{m_i} x_{ij} v_{ij} \quad (7)$$

$$\text{s.t.} \quad \sum_{i=1}^n \sum_{j=1}^{m_i} x_{ij} w_{ij} \leq W \quad (8)$$

$$\sum_{j=1}^{m_i} x_{ij} = 1, \forall i \in \{1, \dots, n\} \quad (9)$$

There is an $O(NW)$ algorithm that solves this problem exactly using dynamic programming [13]. When the objective is to minimize the total profit, we get the minimization form of MCKP, which can be transformed into an equivalent maximization problem [13].

2.4 Machine-learning models

Two machine-learning (ML) models, logistic regression (LR), and neural network (NN) are used in this paper to learn the k NN distributions of queries. We briefly introduce the background knowledge of LR and NN in this section.

Logistic Regression (LR) LR is a special case of generalized linear models [23]. It is a popular method to predict a categorical response. LR is typically used for binary classification. Given an input vector $\mathbf{x}$, LR predicts the possibility that the sample $\mathbf{x}$ belongs to the positive class by $\sigma(\mathbf{w}\mathbf{x} + \mathbf{b})$, where σ is the *logistic* function of $\sigma(z) = \frac{1}{1+e^{-z}}$ (also known as *sigmoid*), and $\mathbf{w}$ and $\mathbf{b}$ are optimizable parameters of the model. Let $y \in \{0, 1\}$ be the ground-truth class label of $\mathbf{x}$, $\hat{y}$ be the output probability of the classifier. The logarithmic loss function is used in the optimization procedure of seeking the best model parameters. The logarithmic loss is defined as

$$\mathcal{L}(\hat{y}, y) = -(y \ln \hat{y} + (1 - y) \ln(1 - \hat{y}))$$

Multinomial logistic regression (also known as softmax regression) is a simple extension of binary logistic regression that works for more than two classes. It uses the *softmax* function instead of *sigmoid* function to predict the outcome probability of multiple classes. For a d -dimensional vector $\mathbf{z}$, the *softmax* is defined as

$$\text{softmax}(\mathbf{z}) = \left[\frac{e^{z_1}}{\sum_{i=1}^d e^{z_i}}, \frac{e^{z_2}}{\sum_{i=1}^d e^{z_i}}, \dots, \frac{e^{z_d}}{\sum_{i=1}^d e^{z_i}} \right]$$

Furthermore, for d classes and a sample $\mathbf{x}$, let $\mathbf{y}$ be the one-hot representation (which means that if $\mathbf{x}$ belongs to the i th class, the i th component of $\mathbf{y}$ is 1 and the others are 0) of the ground-truth class label of $\mathbf{x}$. And $\hat{\mathbf{y}}$ is the output probability of the classifier, the cross-entropy loss is typically used to train the softmax regression model, which is defined as

$$\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = -\sum_{i=1}^d y_i \ln \hat{y}_i$$

Neural Network (NN) Neural networks have attracted widespread attention due to their excellent performance in

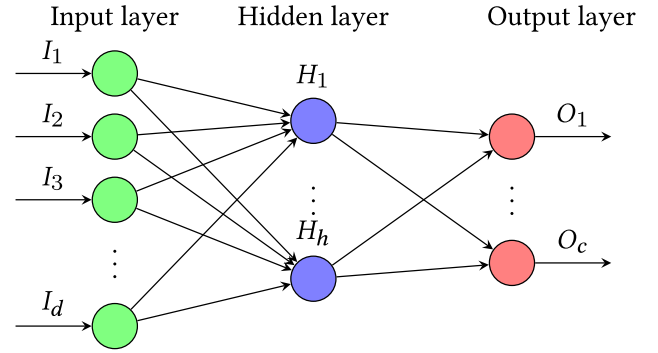


Fig. 3 Illustration of a three-layer neural network

various ML tasks such as classification or regression. Multilayer perceptron (MLP) [16] is one of the most common neural networks. As shown in Fig. 3, an MLP consists of at least three layers of neurons, which are the input layer, the hidden layer, and the output layer. Except for neurons in the input layer, each neuron uses a nonlinear activation function such as *relu* and *sigmoid*. MLPs are typically trained using a supervised learning technique known as backpropagation [8]. MLPs can be used as multi-class classifiers when the output layer is activated by the aforementioned *softmax* function.

3 Motivation

As introduced in Sect. 2.2, the M and c parameters significantly influence both search accuracy and efficiency of multi-probe ANNS. Given a recall requirement r , SOTA multi-probe ANNS approaches leverage a tuning phase to ensure the search recall is close to r (e.g., auto tuning in faiss [29] and recall-aware tuning in ADBV [53]). The tuning process is conducted on a test set formed by some query samples. It will choose proper configurations of M (and c) to ensure the *average* search recall over the test set is no less than r . This procedure makes sense, as it tries to achieve the specified recall target while maximizing the search performance. However, the problem is *all the parameters will be fixed, for each query and each partition* during query execution after tuning. We observe such a fixed-configuration manner brings many redundant computations and leads to a non-optimal accuracy–efficiency trade-off. It focuses on improving the average recall but fully ignores the distinctiveness of queries and partitions.

In this section, we conduct several experiments to evaluate the redundant computation brought by fixed configurations. We use the GLOVE dataset (detailed in Sect. 6.1) in our evaluation. We divide the dataset into 16 partitions through K-means clustering and evaluate the influence of the M and c parameters in multi-probe ANNS approaches.

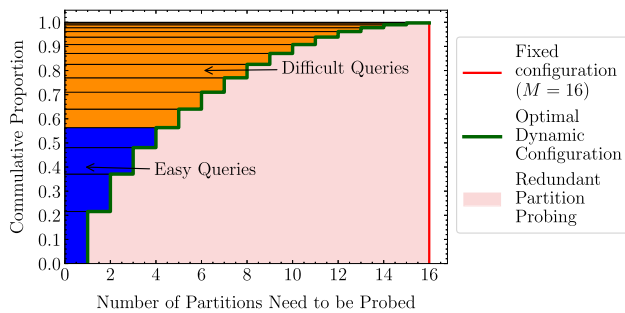


Fig. 4 The minimum number of partitions needs to be probed to reach 100% recall versus commutative proportion of query amounts

3.1 Easy queries versus difficult queries

For each query, we could perform ANNS through probing top- M closest partitions (whose clustering centroids are closest to the query) to find its nearest neighbors. We use M_o to denote the minimum value of M to find the ground-truth 10NNs for a query. For example, a query q 's 10NNs could be found through probing three partitions whose clustering centroids are closest to q , then we have $M_o = 3$ for q . We run 10k queries and collect the value of M_o for each query. Figure 4 plots M_o vs. cumulative proportion of queries. We could see that benefiting from the data locality of clustering-based partitioning, more than 50% “easy” queries only need to probe 1~4 of partitions to find the ground-truth 10NNs of them. However, the other “difficult” queries must probe much more partitions, and in the worst case, all the 16 partitions need to be probed.

Since M_o could be considered as the optimal configuration of M (to achieve 100% recall) and M_o varies among different queries, using a fixed M configuration for all queries is inefficient. It ignores the distinctiveness of queries and brings a significant number of redundant partition probing.

3.2 Light partitions versus heavy partitions

Different queries have different “difficulties” and thus have different optimal M configurations. Similarly, in two-level multi-probe ANNS approaches, a query's optimal configuration of c also varies among different partitions, as a query's k NNs are not uniformly distributed among partitions.

Since we partition the dataset through clustering, when a partition's centroid is closer to the query point than others, this partition probably contains more k NNs of this query point. Let K_i denote the number of a query's k NNs placed in the i th nearest partition ordered by the distance between the partition centroid and the query. We compute values of K_i for 10k queries and plot the distribution of K_i among different partitions in Fig. 5. Obviously, queries' k NNs are

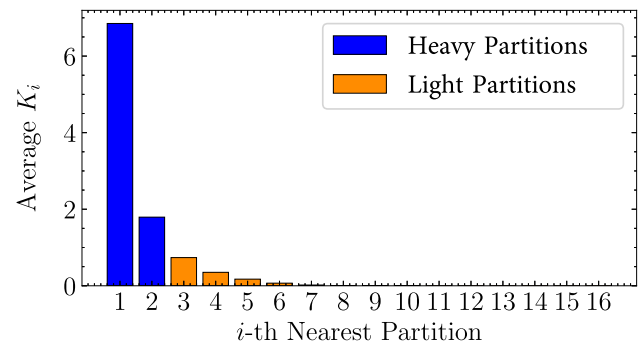


Fig. 5 The distribution of K_i (k NNs placed in the i th nearest partition) among different partitions

not uniformly distributed among partitions. On average, the heaviest partition has more than six items among the ground-truth 10NNs. However, other “light” partitions may have no more than one ground-truth 10NNs. As described in Sect. 2.2, for a query q , a partition containing a small number of q 's k NNs (i.e., having a small K_i value) only needs a small c configuration to find q 's K_i NNs. So, using a fixed c configuration in all partitions is inefficient, as it brings excessive neighbor searching operations for those “light” partitions.

3.3 Opportunities

Fixed configuration-based multi-probe ANNS approaches ignore the distinctiveness of different queries and different partitions, so they could only achieve high recall through redundant partition probing and excessive neighbor searching operations. In other words, they sacrifice too much search efficiency to achieve high accuracy and have a non-optimal accuracy–efficiency trade-off. These observations give us opportunities to further optimize multi-probe ANNS and lead to the following important question: *Can we avoid those redundant computations caused by fixed configurations and achieve optimal accuracy–efficiency trade-off, i.e., satisfy a given recall requirement with minimum cost for multi-probe ANNS?*

4 Problem definition

To overcome the drawbacks of fixed configurations, our basic idea is to generate desirable and efficient search configurations for each query and each partition individually. Given a user-defined recall requirement, for each query, we aim to meet the recall requirement while minimizing the search cost. In the following, we first formulate the multi-probe ANNS optimization problems, then we prove that these problems

Table 1 Frequently used notations

Notation	Definition
1-MAO	One-Level Multi-Probe ANNS Optimization Problem
2-MAO	Two-Level Multi-Probe ANNS Optimization Problem
$\mathbf{q}$	A query vector.
r	The user-defined recall requirement.
N	Total number of partitions.
P_i	The i th partition.
$ P_i $	Data amount of P_i .
b_i	A binary value $\in \{1, 0\}$ which indicates if P_i needs to be probed or not.
c_i	The parameter c_i for P_i in two-level approaches. We have $1 \leq c_i \leq P_i $.
$f_i(\mathbf{q}, k)$	The number of $\mathbf{q}$'s k NNs placed in P_i .
$g_i(\mathbf{q}, P_i , c_i)$	The running time of searching the graph index of P_i with parameter c_i .
$h_i(\mathbf{q}, f_i(\mathbf{q}, k), c_i)$	The number of how many items among the $f_i(\mathbf{q}, k)$ nearest neighbors placed in P_i could be found through searching the graph index of P_i with parameter c_i .

could be transformed into well-studied 0-1 knapsack problem variants and show the practical challenges.

4.1 Problem formulation

Table 1 lists notations used in our formulations. Let N denote the total number of partitions, r denote the user-defined recall requirement, $\mathbf{q}$ denote a query vector. For each query $\mathbf{q}$, LEQAT seeks configurations of multi-probe ANNS methods that minimize the searching cost while ensuring the recall requirement (no less than r). We define this task as two optimization problems.

One-level multi-probe ANNS Optimization Firstly, the search cost of one-level approaches is determined by the number of visited vectors (distance computations). We introduce a decision variable $b_i \in \{1, 0\}$ to indicate if the i th partition will be probed or not. Thus the search cost is defined as $\sum_{i=1}^N b_i |P_i|$, which represents the total number of visited vectors during probing the selected partitions. Secondly, let $f_i(\mathbf{q}, k)$ denote a function that outputs the number of $\mathbf{q}$'s k NNs placed in P_i . To achieve a recall target no less than r , we must have $\sum_{i=1}^N b_i f_i(\mathbf{q}, k) \geq \lceil kr \rceil$, which means that we need to find at least $\lceil kr \rceil$ ground-truth k NNs through probing the selected partitions. Finally, we define the one-level optimization problem as:

Definition 2 (One-Level Multi-Probe ANNS Optimization (1-MAO) Problem) The 1-MAO problem is to find the optimal b_i with minimum search cost (Eq. 10) while ensuring the overall query recall requirement (Eq. 11):

$$\min_{b_i \in \{0, 1\}} \sum_{i=1}^N b_i |P_i| \quad (10)$$

$$\text{s.t.} \quad \sum_{i=1}^N b_i f_i(\mathbf{q}, k) \geq \lceil kr \rceil \quad (11)$$

Two-level multi-probe ANNS Optimization Firstly, $b_i \in \{1, 0\}$ is defined the same as in 1-MAO. Besides, considering the query time for i th partition is related to both the data cardinality $|P_i|$ and the graph search parameter c_i , we use $g_i(\mathbf{q}, |P_i|, c_i)$ to denote the query time on graph index in P_i . Furthermore, let $f_i(\mathbf{q}, k)$ denote the number of $\mathbf{q}$'s k NNs placed in P_i , and let $h_i(\mathbf{q}, f_i(\mathbf{q}, k), c_i)$ denote the number of items among these $f_i(\mathbf{q}, k)$ nearest neighbors could be found through searching the graph index of P_i with parameter c_i . The total search cost could be represented by

$$\sum_{i=1}^N b_i g_i(\mathbf{q}, |P_i|, c_i).$$

And the recall constraint could be represented by

$$\sum_{i=1}^N b_i h_i(\mathbf{q}, f_i(\mathbf{q}, k), c_i) \geq \lceil kr \rceil.$$

It means finding at least $\lceil kr \rceil$ k NNs in selected partitions with parameters $c_1, c_2, \dots, c_N$. Therefore, we define the two-level optimization problem as:

Definition 3 (Two-Level Multi-Probe ANNS Optimization (2-MAO) Problem) The 2-MAO problem is to find the optimal b_i and c_i with minimum search cost (Eq. 12), while ensuring the overall query recall is no less than r (Eq. 13):

$$\min_{b_i \in \{0, 1\}, c_i \in [1, |P_i|]} \sum_{i=1}^N b_i g_i(\mathbf{q}, |P_i|, c_i) \quad (12)$$

$$\text{s.t. } \sum_{i=1}^N b_i h_i(\mathbf{q}, f_i(\mathbf{q}, k), c_i) \geq \lceil kr \rceil \quad (13) \quad \Leftrightarrow \quad \sum_{i=1}^N K_i - \sum_{i=1}^N (1 - x_i) K_i \leq k - v \quad (19)$$

4.2 Analysis and challenges

Solving the problems defined in Definitions 2 and 3 is essential for achieving optimal accuracy–efficiency trade-offs in multi-probe ANNS, i.e., satisfying a given recall requirement through minimum search cost. We present several theoretical analysis pieces and show the practical challenges of solving the 1-MAO and 2-MAO problems in this section.

Firstly, we find that they could be transformed into the well-studied 0–1 knapsack problem and its variants. Formally, we present and prove the following theorems.

Theorem 1 *The 1-MAO problem could be transformed into the 0–1 knapsack problem [13].*

Proof of Theorem 1 $|P_i|$ in Definition 2 is a constant value. Besides, once a query vector is given, the value of $f_i(\mathbf{q}, k)$ is also uniquely determined, thus becomes a constant value. Let $n_i = |P_i|$, $K_i = f_i(\mathbf{q}, k)$, $v = \lceil kr \rceil$, the 1-MAO problem is rewritten as:

$$\min_{b_i \in \{0,1\}} \sum_{i=1}^N b_i n_i \quad (14)$$

$$\text{s.t. } \sum_{i=1}^N b_i K_i \geq v \quad (15)$$

We introduce a new decision variable $x_i = 1 - b_i$, thus the optimization objective

$$\min_{b_i \in \{0,1\}} \sum_{i=1}^N b_i n_i$$

is equivalent to

$$\max_{x_i \in \{0,1\}} \sum_{i=1}^N x_i n_i.$$

Moreover, since $\sum_{i=1}^N K_i = k$, we have:

$$\sum_{i=1}^N b_i K_i \geq v \quad (16)$$

$$\Leftrightarrow k - \sum_{i=1}^N b_i K_i \leq k - v \quad (17)$$

$$\Leftrightarrow k - \sum_{i=1}^N (1 - x_i) K_i \leq k - v \quad (18)$$

$$\Leftrightarrow \sum_{i=1}^N x_i K_i \leq k - v \quad (20)$$

Finally, let $W = k - v$, the 1-MAO problem is rewritten as:

$$\max_{x_i \in \{0,1\}} \sum_{i=1}^N x_i n_i \quad (21)$$

$$\text{s.t. } \sum_{i=1}^N x_i K_i \leq W \quad (22)$$

In practice we have $n_i > 0$ and $0 \leq K_i \leq k$. Obviously, partitions containing no ground-truth kNNs must be dropped, i.e., $K_i = 0 \Rightarrow x_i = 1$, so we only need to consider partitions with $K_i > 0$. Let N' denote the number of nonzero elements in $\mathbf{K} = (K_1, \dots, K_N)$, I_j denote the index of the j th ($1 \leq j \leq N'$) nonzero element in $\mathbf{K}$. For example, for $\mathbf{K} = (0, 6, 4, 0)$, since the index of the first two nonzero elements in $\mathbf{K}$ are 2 and 3, we have $I_1 = 2$ and $I_2 = 3$. Moreover, the corresponding x_i , n_i and K_i values of these partitions having $K_i > 0$ could be represented by $x'_j = x_{I_j}$, $n'_j = n_{I_j}$, and $K'_j = K_{I_j}$. The above problem is rewritten as:

$$\max_{x'_j \in \{0,1\}} \sum_{j=1}^{N'} x'_j n'_j \quad (23)$$

$$\text{s.t. } \sum_{j=1}^{N'} x'_j K'_j \leq W \quad (24)$$

$$K'_j > 0 \quad (25)$$

$$j \in \mathbb{N}^+, 1 \leq j \leq N' \quad (26)$$

This is the standard form of the 0–1 knapsack problem, and there is a dynamic programming (DP) algorithm solving it with time complexity $O(N'W)$ [13]. $\square$

Theorem 2 *The 2-MAO problem could be transformed into the multiple-choice knapsack problem (MCKP) [13,31].*

Proof of Theorem 2 Let $n_i = |P_i|$ denote the data amount of the i th partition, which is a constant value. Besides, once a query $\mathbf{q}$ is given, $f_i(\mathbf{q}, k)$ also becomes a constant. Similarly, $g_i(\mathbf{q}, |P_i|, c_i)$ and $h_i(\mathbf{q}, f_i(\mathbf{q}, k), c_i)$ are uniquely determined by c_i . We use $K_i = f_i(\mathbf{q}, k)$, $G_i(c_i) = g_i(\mathbf{q}, |P_i|, c_i)$ and $H_i(c_i) = h_i(\mathbf{q}, f_i(\mathbf{q}, k), c_i)$ to denote these values for simplicity. Let $v = \lceil kr \rceil$, the 2-MAO problem becomes:

$$\min_{b_i \in \{0,1\}, c_i \in [1, |P_i|]} \sum_{i=1}^N b_i G_i(c_i) \quad (27)$$

$$\text{s.t. } \sum_{i=1}^N b_i H_i(c_i) \geq v \quad (28)$$

Since $0 \leq H_i(c_i) \leq K_i$ and $\sum_{i=1}^N K_i = k$, we have

$$\sum_{i=1}^N b_i H_i(c_i) \geq v \quad (29)$$

$$\Leftrightarrow k - \sum_{i=1}^N b_i H_i(c_i) \leq k - v \quad (30)$$

$$\Leftrightarrow \sum_{i=1}^N K_i - \sum_{i=1}^N b_i H_i(c_i) \leq k - v \quad (31)$$

$$\Leftrightarrow \sum_{i=1}^N (K_i - b_i H_i(c_i)) \leq k - v \quad (32)$$

Let $W = k - v = k - \lceil kr \rceil$, thus the 2-MAO problem could be rewritten as:

$$\min_{x_i \in \{0,1\}, c_i \in [1, n_i]} \sum_{i=1}^N b_i G_i(c_i) \quad (33)$$

$$\text{s.t. } \sum_{i=1}^N (K_i - b_i H_i(c_i)) \leq W \quad (34)$$

For each partition P_i , we need to choose an appropriate value of $b_i \in \{0, 1\}$ and an appropriate value of $c_i \in \{0, 1, \dots, n_i\}$ to solve this problem. In fact, for a partition P_i , we only need to consider $n_i + 1$ possible combinations of b_i and c_i :

1. The first possibility is $b_i = 0$, i.e., do not probe P_i ;
2. The left n_i possibilities are $b_i = 1$ and choosing a value c_i from $\{1, 2, \dots, n_i\}$, i.e., probing P_i with a parameter $c_i \in \{1, 2, \dots, n_i\}$.

We introduce a set

$$S_i = \{s_{i1}, s_{i2}, \dots, s_{i(n_i+1)}\} = \{1, 2, \dots, n_i + 1\}$$

to represent these possibilities. Here, $s_{im} = 1$ represents $b_i = 0$, and $s_{im} = m$, $\forall m \in \{2, 3, \dots, n_i + 1\}$ represents $b_i = 1$ and $c_i = m - 1$. Choosing a value from S_i is equivalent to choosing a corresponding combination of b_i and c_i . For example, choosing $s_{i2} = 2$ is equivalent to set $b_i = 1$ and $c_i = 1$.

Moreover, we introduce new binary decision variables

$$x_{im} \in \{0, 1\}, \text{ where } m \in S_i$$

that take on value 1 if and only if the m th choice in S_i (i.e., s_{im}) is chosen. And we use the constraint

$$\sum_{m \in S_i} x_{im} = 1, \forall i \in \{1, \dots, N\}$$

to ensure only one choice in S_i will be chosen.

Then, we use t_{im} to denote the searching cost $b_i G_i(c_i)$ determined by b_i and c_i , and we have:

$$t_{im} = \begin{cases} 0 & m = 1 \\ G_i(m - 1) & m \in \{2, 3, \dots, n_i + 1\} \end{cases}$$

Similarly, we use w_{im} to denote the values of $K_i - b_i H_i(c_i)$ determined by b_i and c_i , and we have:

$$w_{im} = \begin{cases} 0 & m = 1 \\ K_i - H_i(m - 1) & m \in \{2, 3, \dots, n_i + 1\} \end{cases}$$

So the objective $\sum_{i=1}^N b_i G_i(c_i)$ is equivalent to

$$\sum_{i=1}^N \sum_{m \in S_i} x_{im} t_{im}$$

And we have

$$\sum_{i=1}^N K_i - b_i H_i(c_i) \leq W \quad (35)$$

$$\Leftrightarrow \sum_{i=1}^N \sum_{m \in S_i} x_{im} w_{im} \leq W \quad (36)$$

Therefore, the 2-MAO problem is equivalent to:

$$\min_{x_{im} \in \{0,1\}} \sum_{i=1}^N \sum_{m \in S_i} x_{im} t_{im} \quad (37)$$

$$\text{s.t. } \sum_{i=1}^N \sum_{m \in S_i} x_{im} w_{im} \leq W \quad (38)$$

$$\sum_{m \in S_i} x_{im} = 1, \forall i \in \{1, \dots, N\} \quad (39)$$

In practice, we only need to consider partitions containing ground-truth k NNs, i.e., partitions having $K_i > 0$. Let N' denote the number of nonzero elements in $\mathbf{K} = (K_1, K_2, \dots, K_N)$, I_j denote the index of the j th ($1 \leq j \leq N'$) nonzero element in $\mathbf{K}$. The corresponding S_i , x_{im} , t_{im} , and w_{im} could be represented by $S'_j = S_{I_j}$, $x'_{j m} = x_{I_j m}$, $t'_{j m} = t_{I_j m}$, and $w'_{j m} = w_{I_j m}$. And the above problem could

be rewritten as:

$$\min_{x'_{jm} \in \{0,1\}} \sum_{j=1}^{N'} \sum_{m \in S'_j} x'_{jm} t'_{jm} \quad (40)$$

$$\text{s.t.} \quad \sum_{j=1}^{N'} \sum_{m \in S'_j} x'_{jm} w'_{jm} \leq W \quad (41)$$

$$\sum_{m \in S'_j} x'_{jm} = 1, \forall j \in \{1, \dots, N'\} \quad (42)$$

This problem is known as the minimization form of the multiple-choice knapsack problem (MCKP) [31]. Let $u = \sum_{j=1}^{N'} |S'_j|$, there is a DP algorithm solving it with time complexity $O(uW)$ [13,31]. Recall that $S'_j = S_{I_j} = \{1, \dots, n_{I_j} + 1\}$, where I_j is the index of j th nonzero element in $\mathbf{K}$, thus $|S'_j| = 1 + n_{I_j}$ and $u = \sum_{j=1}^{N'} |S'_j| = \sum_{j=1}^{N'} (1 + n_{I_j}) = N' + \sum_{j=1}^{N'} n_{I_j}$. Here $\sum_{j=1}^{N'} n_{I_j}$ is the data amount of partitions whose $K_i > 0$, i.e., data amount of partitions containing ground-truth k NNs of the given query. Let $n' = \sum_{j=1}^{N'} n_{I_j}$, we have $u = N' + n'$, and time complexity of the DP algorithm becomes $O((N' + n')W)$. $\square$

Analysis By transforming our problems to the well-studied knapsack-problem variants, we could use existing algorithms to find theoretical optimal solutions for them. For a k NN query $\mathbf{q}$, we only need to consider partitions containing $\mathbf{q}$'s ground-truth k NNs. We use N' to denote the number of partitions containing $\mathbf{q}$'s k NNs and use n' to denote these partitions' data amount. Besides, let $W = k - \lceil kr \rceil$. There exists a dynamic programming (DP) solution for the 1-MAO problem with time complexity $O(N'W)$. Similarly, there exists a DP algorithm solving the 2-MAO problem with time complexity $O((N' + n')W)$. However, to build a practical approach, there are still several challenges that need to be addressed.

Challenge1: *The k NN distribution information is essential for solving the two optimization problems, but it is unknown before the query execution* Let $K_i = f_i(\mathbf{q}, k)$ denote the number of $\mathbf{q}$'s k NNs placed in partition P_i . Essentially, the vector $\mathbf{K} = (K_1, K_2, \dots, K_N)$ indicates how $\mathbf{q}$'s k NNs distributed across the N partitions. By normalizing $\mathbf{K}$, we get a probability distribution vector $\mathbf{y} = (\frac{K_1}{\sum_i K_i}, \dots, \frac{K_N}{\sum_i K_i})$, where $\sum_i K_i = k$, $0 \leq y_i \leq 1$ and $\sum_i y_i = 1$. Thus we call $\mathbf{y}$ the k NN distribution vector and we formally define the k NN distribution vector as:

Definition 4 (*k NN Distribution Vector*) Given a N -partition dataset and a query vector $\mathbf{q}$, let $K_i = f_i(\mathbf{q}, k)$ denote the number of $\mathbf{q}$'s k NNs placed in the i th partition, $\mathbf{q}$'s k NN distribution vector is defined as $\mathbf{y} = (\frac{K_1}{k}, \dots, \frac{K_N}{k})$, which is

a probability distribution vector whose elements are in $[0, 1]$ and sum up to 1.

We notice that $f_i(\mathbf{q}, k)$ appears in both Definitions 2 and 3. It means that the k NN distribution information is essential for solving both the 1-MAO and 2-MAO problems. However, queries' k NN distributions are unknown until the query execution, solving the two optimization problems thus becomes a non-trivial task. In this paper, we propose the novel k NN distribution estimation technique (detailed in Sect. 5.2) to address this issue and solve the optimization problems.

Challenge2: *The exact algorithm for 2-MAO leads to unacceptable computation cost* We have shown that there is an algorithm solving 1-MAO with $O(N'W)$ complexity. Here N' is the number of partitions containing a query's k NNs and $W = (k - \lceil kr \rceil)$. In practice, both N' and W would not be too large (e.g. $k = 10$, $r = 0.9$, $N' = 5$, and $W = (10 - (10 * 0.9)) = 1$), which means we could solve the 1-MAO problem efficiently. However, for the 2-MAO problem, finding a practical solution becomes more complicated. Firstly, we observe $g_i(\mathbf{q}, |P_i|, c_i)$ and $h_i(\mathbf{q}, f_i(\mathbf{q}, k), c_i)$ appear in the 2-MAO definition (Eqs. 12 and 13). To solve the problem, we must predict $g_i(\cdot)$ and $h_i(\cdot)$ for queries before execution. Secondly, even if we would find accurate predictions of $g_i(\cdot)$ and $h_i(\cdot)$, a practical dynamic programming algorithm for 2-MAO has $O((N' + n')W)$ complexity, where n' is the data amount of the N' partitions containing ground-truth k NNs of the given query. Suppose that the k NN distribution of a given query $\mathbf{q}$ is known exactly, we could find the exact k NNs through visiting these n' vectors in these N' partitions containing $\mathbf{q}$'s k NNs, i.e., performing n' times of distance computations. However, the complexity of the practical DP algorithm for solving 2-MAO is $O((N' + n')W)$. Such high computation cost is unacceptable in practice, as the optimization overhead could even exceed the query execution time. To address this challenge, in Sect. 5.4, we introduce several heuristic simplifications of the 2-MAO problem and then propose algorithms to solve the simplified problem efficiently.

5 Algorithms

5.1 Overview

We propose LEQAT (LEarned Query-Aware OpTimizer) to address those problems and challenges described in Sect. 4. Essentially, LEQAT is a per-query optimization framework for multi-probe ANNS. It is designed to achieve optimal accuracy-efficiency trade-off for one-level and two-level multi-probe ANNS methods. LEQAT has practical and efficient algorithms for solving the 1-MAO and 2-MAO problems. It takes a query and a recall requirement r as input

and outputs an efficient, customized configuration for the query while satisfying the recall requirement. The design of LEQAT includes the following two critical parts:

1. *kNN distribution estimation* We have demonstrated the importance of k NN distribution in multi-probe ANNS optimization. We propose novel k NN distribution estimation techniques in LEQAT to efficiently and accurately predict queries' k NN distributions before the optimization.
2. *Query-aware search configuration* LEQAT also consists of practical algorithms for solving the 1-MAO and 2-MAO problems. It generates configurations for each individual query through solving the two problems before query execution. For the 1-MAO problem, we use DP to find the optimal solution. For the more complicated 2-MAO problem, we propose efficient heuristic algorithms to find practical solutions.

5.2 k NN distribution estimation

We build estimators to predict queries' k NN distribution vectors. For each query q , the estimator predicts q 's k NN distribution vector y before query execution. The design of our estimators follows the following two principles:

1. *Accurate* The 1-MAO and 2-MAO problems both assume that the k NN distribution of a query is known precisely. An estimator must produce reasonable accurate predictions to help us solve the problems.
2. *Efficient* We cannot afford too large overhead in the k NN distribution estimation phase, i.e., the estimation must be much faster than the query execution.

In this section, we propose two types of models to estimate queries' k NN distributions guided by the above principles.

A naive approach (NA) Given a query vector q , for partitions produced by K-means clustering, we could get the distances between q and the clustering centroids of these N partitions. Let d_i denote the distance between q and P_i 's clustering centroid, and $\mathbf{d}$ represent the vector $(d_1, d_2, \dots, d_N)$. Based on the intuition that a closer partition may contain more nearest neighbors, we use $\hat{\mathbf{y}} = \text{softmax}(-\mathbf{d})$ as a coarse-grained prediction for q 's k NN distribution vector. NA is a naive baseline to show the superiority of learning-based models.

Learning-based approaches Though NA is efficient, it cannot make accurate k NN distribution predictions in high-dimensional space. On the other hand, query histories are usually available in practice. They have been successfully used to train supervised machine learning models to improve the speed of future query execution of nearest neighbors search in various researches [2,42,51]. Therefore, we propose

Algorithm 1: dp-pruning($k, r, K[1 \dots N], PC[1 \dots N]$)

Input : k for top- k search; r : a recall requirement; $K[1 \dots N]$: number of k NNs placed in each partition; $PC[1 \dots N]$: data amount of each partition.

Output: $b[1 \dots N]$, where $b[i]$ is 1 or 0 and indicates if the i -th partition will be probed or not.

```

1  dp (  $W, K'[1 \dots N'], PC'[1 \dots N']$  ) initialize  $x[1 \dots N']$ ;
2  find optimal  $x[1 \dots N']$  through solving the 0-1 knapsack
   problem defined in Equation 23~26 through dynamic
   programming;
3  return  $x[1 \dots N']$ ;
4  initialize  $b[1 \dots N]$  with zeros;
5   $W \leftarrow k - \lceil kr \rceil$ ;
   /*  $l$  is the id list of partitions
   containing the given query's  $k$ NNs */
6   $l \leftarrow$  an empty list ;
7   $K' \leftarrow$  an empty list;
8   $PC' \leftarrow$  an empty list;
9  for  $i \leftarrow 1$  to  $N$  do
10     if  $K[i] = 0$  then
11          $b[i] \leftarrow 0$ ;
12     else
13         append  $i$  to the end of  $l$ ;
14         append  $PC[i]$  to the end of  $PC'$ ;
15         append  $K[i]$  to the end of  $K'$ ;
16   $N' \leftarrow$  length of  $l$ ;
17   $x[1 \dots N'] \leftarrow dp(W, K', PC')$ ;
18  for  $j \leftarrow 1$  to  $N'$  do
19      $I \leftarrow l[j]$ ;
20      $b[I] \leftarrow 1 - x[j]$ ;
21  return  $b[1 \dots N]$ ;
```

to use learning-based approaches to estimate k NN distribution for our 1-MAO and 2-MAO problems. We treat the N partitions of a dataset as N classes and view the k NN distribution vectors as soft labels. Then, the well-studied multi-class classification models could be naturally used as k NN distribution estimators. In this paper, we consider two machine learning models that work with soft labels and can predict probability distributions of output classes: (i) logistic regression (LR) and (ii) neural network (NN).

5.3 Query-aware search configuration for one-level approaches

In this section, we introduce our practical algorithm for solving the 1-MAO problem, which can generate efficient search configurations for individual queries.

We have shown there is an efficient dynamic programming approach to solve 1-MAO exactly. The algorithm is outlined in Algorithm 1. Given k, r (the user-defined recall requirement), let $PC[i] = |P_i|$ represent the data amount of P_i , and $K[i]$ the number of k NNs placed in P_i . The core idea of the function $dp(\cdot)$ shown in Algorithm 1 is to calculate which partitions should be “dropped” instead of which partitions

should be “selected” for further probing. So we name this algorithm as *dp-pruning*. We do not give more details for Algorithm 1 as it is a standard exact algorithm for the 0-1 knapsack problem. In practice, k is usually small. Thus we could find the optimal solution efficiently.

However, to apply *dp-pruning* in practice, we must also consider the prediction error introduced by k NN distribution estimation, because the user-defined recall requirement r may not be satisfied by *dp-pruning* due to k NN distribution estimation errors. The SOTA ANNS approaches cannot provide theoretical recall guarantees. Moreover, in the worst case, one has to search all the partitions to meet the recall requirement. Therefore, in practice, the recall requirement r defined by users is also an approximate value, and different heuristics are used so that the average recall can approximate r , e.g., recall-aware tuning in ADBV [53] and auto tuning in Faiss [29]. Similarly, we introduce a flexible probability threshold σ as a hyperparameter to compensate for the effects introduced by prediction errors. Specifically, we use σ as a recall constraint in *dp-pruning* instead of using r directly. Intuitively, a larger σ will result in a higher probability of satisfying recall r , but a more expensive configuration. We tune σ by executing a set of validation queries in the tuning phase and ensure that the average search recall is no less than r . More details of the tuning will be discussed in Sect. 5.5.

Besides, *dp-pruning* works for integer values of K_i instead of decimal values of y_i . Therefore, to reduce numerical precision loss, for $k < 1000$ we introduce an enlarge factor $\alpha = \max(1, \lfloor \frac{1000}{k} \rfloor)$ and cast $\hat{\mathbf{y}}$ to integers by $\text{round}(\alpha k \hat{\mathbf{y}})$. This ensures at least 3 decimal digits of $\hat{\mathbf{y}}$'s elements are preserved.

Finally, let $\alpha = \max(1, \lfloor \frac{1000}{k} \rfloor)$, we solve 1-MAO by *dp-pruning*($\alpha k, \sigma, \text{round}(\alpha k \hat{\mathbf{y}}), \dots$).

5.4 Query-aware search configuration for two-level approaches

Heuristic Simplifications As described in Sect. 4.2, the exact algorithm of 2-MAO needs the exact values of $h_i(\cdot)$ and $g_i(\cdot)$ and is computationally prohibitive. Therefore, we introduce the following heuristics to simplify the original 2-MAO problem and propose efficient algorithms solving the reduced problem:

1. Given that a larger c_i or $|P_i|$ leads to higher search latency, we use the weighted sum of normalized c_i and $|P_i|$ to represent the search cost $g_i(\mathbf{q}, |P_i|, c_i)$. More specifically, we firstly normalize c_i and $|P_i|$ to the range of 0 to 1 by dividing the sum of c_i or $|P_i|$ in all partitions. Then, let $\tilde{c}_i$ and $|\tilde{P}_i|$ denote the normalized values, we introduce the hyperparameter $\beta \in [0, 1]$ to balance the impact of $\tilde{c}_i$ and $|\tilde{P}_i|$, and we use the weighted sum to represent the search

cost t_i for the i th partition:

$$t_i = \beta \tilde{c}_i + (1 - \beta) |\tilde{P}_i|$$

2. Because a larger k needs a larger c to ensure accuracy, we introduce a flexible hyperparameter δ to set c_i as $\delta f_i(\mathbf{q}, k)$, where $f_i(\mathbf{q}, k)$ is the number of $\mathbf{q}$'s k NNs in P_i . That ensures a partition containing less ground-truth k NNs has a smaller c configuration.

Therefore, the original 2-MAO problem is simplified as:

$$\min_{b_i \in \{0,1\}} \sum_{i=1}^N b_i t_i \quad (43)$$

$$\text{s.t.} \quad \sum_{i=1}^N b_i f_i(\mathbf{q}, k) \geq \lceil kr \rceil \quad (44)$$

$$t_i = \beta \tilde{c}_i + (1 - \beta) |\tilde{P}_i| \quad (45)$$

$$c_i = \delta f_i(\mathbf{q}, k) \quad (46)$$

$$\tilde{c}_i = \frac{c_i}{\sum_{i=1}^N c_i} \quad (47)$$

$$|\tilde{P}_i| = \frac{|P_i|}{\sum_{i=1}^N |P_i|} \quad (48)$$

Dynamic Programming Algorithm The simplified 2-MAO problem in Eqs. 43–48 is similar to the 1-MAO problem defined in Eqs. 10 and 11. More specifically, by replacing $|P_i|$ in Eq. 10 with t_i , we get the simplified version of 2-MAO. That means we can use the same techniques and algorithms described in Sect. 5.3 to efficiently find solutions to the simplified 2-MAO problem:

- (i) To ensure recall requirement, we introduce a flexible hyperparameter τ to replace r .
- (ii) To avoid numerical precision loss, we enlarge k to αk and compute $f_i(\mathbf{q}, k)$ by $\text{round}(\alpha k \hat{\mathbf{y}})$, where $\alpha = \max(1, \lfloor \frac{1000}{k} \rfloor)$ and $\hat{\mathbf{y}}$ is the estimated k NN distribution vector.
- (iii) Since we have

$$\begin{aligned} \tilde{c}_i &= \frac{c_i}{\sum_{i=1}^N c_i} = \frac{\delta f_i(\mathbf{q}, k)}{\sum_{i=1}^N \delta f_i(\mathbf{q}, k)} \\ &= \frac{\delta f_i(\mathbf{q}, k)}{\sum_{i=1}^N \delta f_i(\mathbf{q}, k)} = \frac{\delta \cdot \text{round}(\alpha k \hat{\mathbf{y}}_i)}{\sum_{i=1}^N \delta \cdot \text{round}(\alpha k \hat{\mathbf{y}}_i)} \\ &\approx \frac{\hat{\mathbf{y}}_i}{\sum_{i=1}^N \hat{\mathbf{y}}_i} = \hat{\mathbf{y}}_i \end{aligned}$$

Therefore, we set $\tilde{c}_i$ as $\hat{\mathbf{y}}_i$ to avoid numerical loss, so that $t_i = \beta \hat{\mathbf{y}}_i + (1 - \beta) |\tilde{P}_i|$. Besides, we preserve three decimal digits of t_i and cast it to an integer by $\text{round}(1000t_i)$.

Table 2 Dataset specifications

Dataset	Vector dimensions	# of Base vectors	# of Query vectors	# of Training vectors	Distance function
GIST [26]	960	1M	10k	–	Euclidean
GLOVE [41]	100	1.18M	1k	–	Angular
SIFT [28]	128	500M	10k	1M	Euclidean
DEEP [5]	96	500M	10k	1M	Angular

Algorithm 2: greedy-pruning($k, r, y[1 \dots N], PC[1 \dots N]$)

Input : k for top- k search; r : a recall requirement; $y[1 \dots N]$: estimated k NN distribution vector; $PC[1 \dots N]$: data amount of each partition.

Output: $b[1 \dots N]$, where $b[i]$ is 1 or 0 and indicates if the i -th partition will be probed or not.

```

1  $ids \leftarrow \{1, 2, \dots, N\}$ ;
2 sort elements in  $ids$  by their corresponding  $k$ NN distribution
  probabilities in  $y[1 \dots N]$  in descending order;
3  $sum \leftarrow 0$ ;
4 initialize  $b[1 \dots N]$  with zeros;
5 foreach  $i$  in  $ids$  do
6    $b[i] \leftarrow 1$ ;
7    $sum \leftarrow sum + y[i]$ ;
8   if  $sum \geq r$  then
9     break;
10 return  $b[1 \dots N]$ 
```

Finally, let $\alpha = \max(1, \lfloor \frac{1000}{k} \rfloor)$, we solve the simplified 2-MAO problem by dp -pruning($\alpha k, \tau, \text{round}(\alpha k \hat{y}), \{\text{round}(1000t_1), \dots, \text{round}(1000t_N)\}$). The function dp -pruning determines the partitions to be probed. In addition, as shown in Eq. 46, the graph search parameter c_i is set as $c_i = \delta f_i(q, k) = \delta \cdot \text{round}(\alpha k \hat{y})$. Note that β in Eq. 45, as well as τ and δ described earlier, are three hyperparameters will be tuned by LEQAT.

Greedy Algorithm The fixed-configuration approach searches M nearest partitions with parameter c . Since a closer partition typically has more ground-truth k NNs (as shown in Sect. 3.2), such an approach can be considered as a greedy algorithm that always visits the partition that may have the most ground-truth k NNs. However, because the k NN distribution is unknown, this approach uses fixed configurations for all queries thus are inefficient. Beneficial from our novel k NN distribution estimation technique, we design a similar but more advance greedy algorithm to solve the 2-MAO problem.

Our algorithm is outlined in Algorithm 2 and is named *greedy-pruning*. For a specific query, it firstly sorts the partitions by the estimated k NN distribution vector instead of the partition centroid distance. It then iteratively selects the partition having the largest value in the k NN distribution vector (i.e., having the most ground-truth k NNs) until the total recall is no less than the pre-defined requirement r . In practice, we

use a flexible recall threshold τ to replace r . Furthermore, to ensure the partition having less ground-truth k NNs has a smaller c configuration, we search each selected partition with parameter of $c_i = \text{round}(\delta k \hat{y}_i)$, where $\hat{y}_i$ is the i th element in the estimated k NN distribution vector. Both τ and δ are hyperparameters will be tuned and determined by LEQAT.

It is worth noting that we introduce several intuitive simplifications when solving the 2-MAO problem. But we argue that, although our algorithm cannot solve the original 2-MAO problem exactly, we could still achieve a significant speedup (up to 3.9x, detailed in Sect. 6) for two-level multi-probe ANNS approaches. We view this practical performance as one of the main contributions of this paper.

5.5 Recall-aware tuning

Given the user-defined recall requirement r and k , we need to find appropriate hyperparameters in LEQAT: (i) σ for one-level approaches, or (ii) β, δ , and τ for two-level approaches. Otherwise, LEQAT is not able to generate query-aware configurations through solutions described in Sects. 5.3 and 5.4. The hyperparameter tuning process is performed in a preprocessing phase. Firstly, we use 10k historical or sample query vectors as a validation set. Then, we use grid search to enumerate possible combinations of hyperparameters. Among them, we only consider combinations that could achieve average recall larger than r for the 10k validation queries. Then the combination of the lowest execution time will be recorded as the final setting of hyperparameters.

6 Experiments

6.1 Methodology

Evaluated one-level multi-probe ANNS methods We evaluate one-level methods with different combinations of the following components. Two space partitioning techniques: (1) K-means clustering (abbreviated as kmeans) and (2) Cross-polytope LSH [3] (abbreviated as CPLSH). Two search methods: (1) *IVF-Fixed* (probing the nearest M partitions for each query) and (2) LEQAT (using dp -pruning to generate

Table 3 End-to-end search latency over 1024 partitions (partitioned by K-means clustering) under different top-10 recall targets

GIST Top-10 recall target	IVF fixed search parameter (M)	IVF search latency (ms)	LEQAT hyperparameter (σ)	LEQAT search latency (ms)	Latency reduction (%)
0.90	10	25.8357	0.890	20.4129	20
0.92	12	30.8628	0.900	21.5800	30
0.94	15	38.0932	0.930	26.2340	32
0.96	17	43.2006	0.954	32.5035	26
0.98	24	58.8331	0.980	44.5486	24
GLOVE Top-10 recall target					
	IVF fixed search parameter (M)	IVF search latency (ms)	LEQAT hyperparameter (σ)	LEQAT search latency (ms)	Latency reduction (%)
0.90	32	3.020	0.892	2.369	33
0.92	42	4.247	0.910	2.664	50
0.94	46	5.992	0.934	2.345	40
0.96	66	7.424	0.950	4.000	43
0.98	82	12.678	0.975	5.985	58

Table 4 End-to-end search latency over 1024 partitions (partitioned by K-means clustering) under different top-100 recall targets

GIST Top-100 recall target	IVF fixed search parameter (M)	IVF search latency (ms)	LEQAT hyperparameter (σ)	LEQAT search latency (ms)	Latency reduction (%)
0.90	14	48.6121	0.890	37.3634	23
0.92	16	55.4041	0.910	41.753	35
0.94	19	64.8892	0.928	46.6107	28
0.96	23	77.035	0.948	54.3463	30
0.98	31	101.472	0.975	71.3837	24
GLOVE Top-100 recall target					
	IVF fixed search parameter (M)	IVF search latency (ms)	LEQAT hyperparameter (σ)	LEQAT search latency (ms)	Latency reduction (%)
0.90	64	7.6033	0.88	4.9325	43
0.92	80	9.4661	0.90	5.6377	44
0.94	106	12.4989	0.93	7.2537	42
0.96	150	17.7121	0.95	9.0129	47
0.98	220	25.8971	0.97	11.8958	56

query-aware configurations). Three k NN distribution estimators in LEQAT: (1) NA (which means the naive approach for k NN distribution estimation described in Sect. 5.2), (2) LR, and (3) NN. We use x -IVF-Fixed to denote IVF-Fixed for partitions produced by x , e.g., kmeans-IVF-Fixed. We use x -LEQAT- y to denote LEQAT with the estimator y for partitions produced by x , e.g., kmeans-LEQAT-NA. Here kmeans-IVF-Fixed, kmeans-LEQAT-NA and CPLSH-IVF-Fixed are considered as baselines.

Evaluated two-level multi-probe ANNS methods We only consider K-means clustering-based partitioning in the evaluation of two-level methods, as its widespread use and excellent performance under distributed settings [29,53]. Moreover, the two state-of-the-art graph-based ANNS indexes HNSW [38] and SSG [14] are evaluated. We use them as local partition indexes and compare the performance of fixed configuration and LEQAT.

Datasets Two million-scale datasets are used for one-level approaches evaluation: GIST and GLOVE (Table 2). Each of them provides about 1M base vectors and a separate set of query vectors. We use all the base vectors and their ground-truth k NNs to train the k NN distribution estimators in LEQAT directly. Two billion-scale datasets are used for two-level approaches evaluation: SIFT and DEEP. We use the first 500M base vectors of them in our evaluation. Each of them also includes a separate training set; we use 1M training vectors to train the k NN distribution estimators in LEQAT.

Hardware System We execute the experiments on a cluster of 8 machines connected with 10Gbps Ethernet (experiments of one-level multi-probe ANNS approaches are executed on a single machine). Each machine is equipped with one Intel Xeon Platinum 8269CY (64 bit) Processor with 32 cores and 128 GB memory. Ubuntu 18.04.4 LTS 64-bit is the operating system.

Implementation Details Firstly, we develop the K-means partitioner with Scipy1.6.2 and Python3.7, and the CPLSH partitioner with the FALCONN¹ library. Secondly, we implement the baseline search algorithms and LEQAT algorithms with C++11. And we use the open-sourced C++ code of HNSW² and SSG³. We also use OpenMPI to perform distributed query processing for two-level approaches. The code is compiled with GCC 7.5.0 enabling the “O3” optimization. Thirdly, we train the logistic regression (LR) and neural network (NN) models in LEQAT with Tensorflow 2.1 and Python 3.7, then load them in C++ with the Intel OpenVINO library for fast inference. More implementation details of these models are presented as follows.

- (1) LR. The LR model in LEQAT is implemented as a single-layer network with Tensorflow. It accepts d inputs and outputs an N -dimensional vector y . Here d is the dimensionality of database vectors, and N is the total number of partitions. We use the *softmax* activation function in the output layer to produce probability distributions. Moreover, we normalize the input vector values to the range of 0 to 1, using the cross-entropy loss and Adam optimizer in Tensorflow to train this model.
- (2) NN. The NN model in LEQAT consists of three layers: the input layer accepting d inputs, the hidden layer containing h hidden units, and the output layer outputting an N -dimensional vector y . We use the *relu* and *softmax* activation function in the hidden and output layer, respectively. h is set as 256 in our evaluation. Moreover, we normalize the input vector values to the range of 0–1, using the cross-entropy loss and Adam optimizer in Tensorflow to train this model.

6.2 k NN distribution estimation accuracy

We evaluate the k NN distribution estimation accuracy in this section. In our evaluation, both NN and LR demonstrate very high estimation accuracy, whose mean absolute error (MAE) varies from 0.005 to 0.02 depending on the dataset type and the number of partitions. NA yields 5–10 times larger MAE comparing with NN and LR. Note that MAE is only an intuitive metric to observe models’ prediction accuracy and cannot be used directly to evaluate models’ practical effects. We detail the evaluation of model’s practical effects for multi-probe ANNS in Sects. 6.4 and 6.5.

6.3 LEQAT optimization overhead

LEQAT generates configurations for each query dynamically, thus introduce optimization overheads. Overall, one prediction of the k NN distribution estimator in LEQAT takes 0.05–0.2 ms, and 1 round of complete optimization in LEQAT for a query takes 0.15–0.3 ms depending on different settings of k (for top- k ANNS search), d (vector dimensionality), and N (total number of partitions). We also evaluate the end-to-end performance of LEQAT, and it turns out that although LEQAT introduces a little optimization overhead, it is still beneficial due to the query execution takes longer time (detailed in Sects. 6.4 and 6.5).

6.4 One-level approaches evaluation

Performance Metrics Recall and latency are used as metrics in our evaluation for one-level approaches. We use recall (computed by Eq. 1) to measure the accuracy of a result set returned by an ANNS algorithm, and we focus on the performance of algorithms in the high-recall range (over 90%)

¹ <https://github.com/FALCONN-LIB/FALCONN>

² <https://github.com/nmslib/hnswlib>

³ <https://github.com/ZJULearning/SSG>

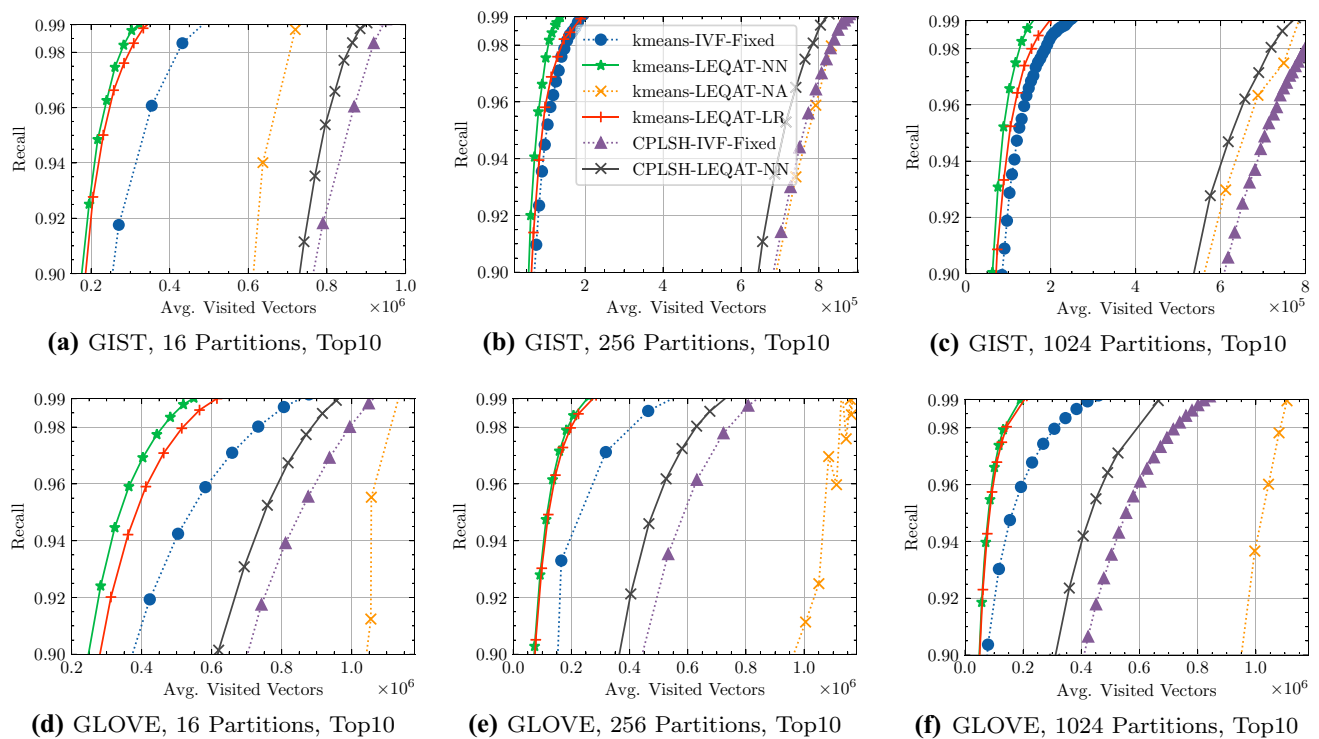


Fig. 6 Effect of partition pruning for one-level approaches

which is required in real scenarios. Moreover, we run test queries one by one and use the average per-query execution time to measure query latency.

Preprocessing For each dataset, we partition it into 16/256/1024 partitions through K-means or CPLSH. Then, we compute the ground-truth k NN distribution vectors for the training set through exhaustive search. Finally, we use 99% training vectors to train the NN or LR model in LEQAT, and the remaining 1% vectors are used as a validation set for early stopping. The model is trained under 50 epochs, and we stop the training procedure when the model's loss on the validation set does not decrease in the recent 10 epochs.

Effect of partition pruning Figure 6 plots the average visited vectors versus recall, comparing IVF-Fixed and LEQAT, which reflects the effect of partition pruning. The recall is tuned by the M parameter of IVF-Fixed and the σ parameter of LEQAT. It turns out that kmeans-IVF-Fixed is a strong-enough baseline as it could achieve high recall through visiting a small proportion of vectors of the whole dataset.

In all settings considered, kmeans-LEQAT-NN outperforms other methods. Besides, for both K-means and CPLSH partitioning, LEQAT could achieve a better efficiency–accuracy trade-off: achieving the same recall, LEQAT reduces up to 60% vector visiting in the K-means case and up to 25% vector visiting in the CPLSH case. It is also worth noting that kmeans-LEQAT-NA performs much worse than kmeans-LEQAT-NN and even worse than the base-

line kmeans-IVF-Fixed. It is mainly because that the k NN distribution estimation error of the NA model misleads the *dp-pruning* algorithm to make wrong decisions. Partitions containing many k NNs may be dropped by *dp-pruning* incorrectly, as their k NN distribution probabilities may be underestimated, which would worsen the overall recall.

End-to-end performance evaluation Tables 3 and 4 summarize the end-to-end search latency of IVF-Fixed and IVF-LEQAT over two million-scale datasets under different recall targets, where each dataset is partitioned to 1024 partitions through K-means. Overall, LEQAT provides consistent latency reduction from 20 to 58% comparing with IVF-Fixed. LEQAT has more significant improvement for GLOVE because GLOVE is a relatively “difficult” dataset for ANNS: it needs probing 82 partitions to reach 98% recall; however, GIST only needs 24. Such difficulty leaves us more space for improvement.

The search latency over GIST is longer than GLOVE due to its higher dimension (960). The accuracy–efficiency trade-off of IVF-Fixed and IVF-LEQAT are tuned by M and σ . Both recall and latency increase as the parameter M of IVF-Fixed and the hyperparameter σ of LEQAT increase. The increase of M leads to more partition probing for all queries as it is fixed for all queries. In contrast, σ is a heuristic hyperparameter, and the increase of σ only leads to more partition probing for those “difficult” queries, thus LEQAT achieves a better accuracy–efficiency trade-off.

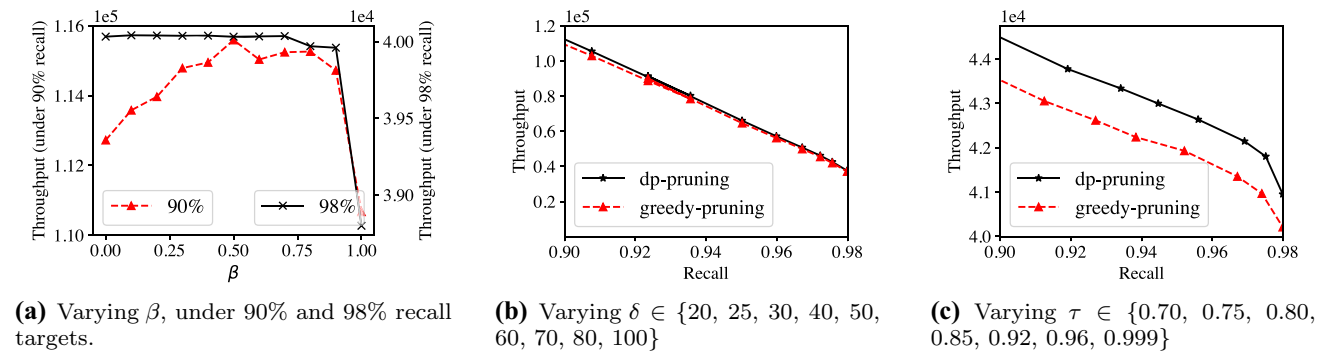


Fig. 7 Hyperparameter study

6.5 Two-level approaches evaluation

Performance Metrics Recall and latency described in Sect. 6.4 are also used in evaluation in this section. Latency is a useful metric in online serving scenarios. Besides, in distributed settings, we also care about the query throughput, which is useful in offline analytic scenarios. So we also evaluate the query throughput of LEQAT and baseline methods in this section. We run all queries in parallel and use the average number of queries executed in a second to measure the query throughput.

Evaluated Methods We apply LEQAT to HNSW and SSG and compare the performance of LEQAT with fixed configurations. We use *X*-Clustering to denote the baseline methods. Here *X* denotes the name of the graph index we are using, and Clustering means that we select fixed *M* partitions whose clustering centroids are closest to the query vector and set a fixed *c* configuration for all partitions. Similarly, we use *X*-LEQAT to refer LEQAT with the graph index *X*.

Preprocessing We set the partition number to 8, 16, and 32 and distribute these partitions to our 8 machines evenly. Graph indexes (both HNSW and SSG) are built for these partitions, and we use the index construction parameters according to the recommendation in the original papers of HNSW[38] and SSG[14] without fine-tuning them. Moreover, we find the top-10 and top-100 nearest neighbors for the 1 million training vectors in each dataset to generate the training input-output pairs required by the LEQAT's *k*NN distribution estimators. This is done through HNSW-Clustering with 99.9% recall configurations. At last, we train the NN model in LEQAT following the same preprocessing methodology described in Sect. 6.4.

Overall Speedup We set the total partition number equal to the machine number and build HNSW indexes to compare the performance of LEQAT and the fixed-configuration approach HNSW-Clustering. Table 5 shows the overall speedup ratio achieved by LEQAT, with the two optimization algorithms *dp-pruning* and *greedy-pruning*. As shown in the table, with *dp-pruning*, LEQAT can consistently achieve significant

speedup up to 2.89x over the fixed-configuration approach, under different recall targets. We also notice that although *greedy-pruning* is 2–10% worse than *dp-pruning* (since *dp-pruning* additionally takes into account the data cardinality of each partition), it can also significantly speed up the search performance up to 2.82x. This is mainly because the graph search is of $O(\log(n))$ time complexity for the data cardinality *n*, resulting in the data cardinality having a limited impact on the overall performance. In contrast, the query-aware configurations of search partitions and the search parameter *c_i* play critical roles in search acceleration. Such configurations significantly avoid redundant computations caused by fixed configurations, so that *greedy-pruning* also achieves considerable speedup (Table 5).

Hyperparameter Study We then use the SIFT500M dataset to evaluate the impacts of the three hyperparameters β , δ , τ when applying *dp-pruning* and the two hyperparameters δ , τ for *greedy-pruning*. Firstly, we fix the recall target around 90% and 98% and evaluate the impact of β for *dp-pruning*. As shown in Fig. 7a, under the 90% recall target, the throughput reaches the maximum when $\beta = 0.5$ and then decreases. It indicates that $\beta = 0.5$ strikes a good balance for impacts from *c_i* and $|P_i|$. As to the high recall target 98%, varying β has limited impacts on the search performance. That is mainly because that β only impacts the partition pruning quality; however, under high recall targets, the number of partitions that need to be probed is typically high so that the speedup gain from tuning β becomes relatively small.

Then, we set $\beta = 0.5$ (only for *dp-pruning*) and $\tau = 0.999$ (for both *dp-pruning* and *greedy-pruning*) to evaluate the search performance under different δ , setting $\beta = 0.5$ and $\delta = 100$ to evaluate the search performance under different τ . As shown in Fig. 7b and c, larger values of δ and τ lead to higher recall but lower search throughput. We can also conclude that the search performance is more sensitive to δ than τ , as the throughput under the worst case is about 60% lower than the best case when varying δ . Therefore, for hyperparameter tuning, we recommend using a large-enough τ (e.g., 0.999) and finding a proper δ to satisfy the recall requirement,

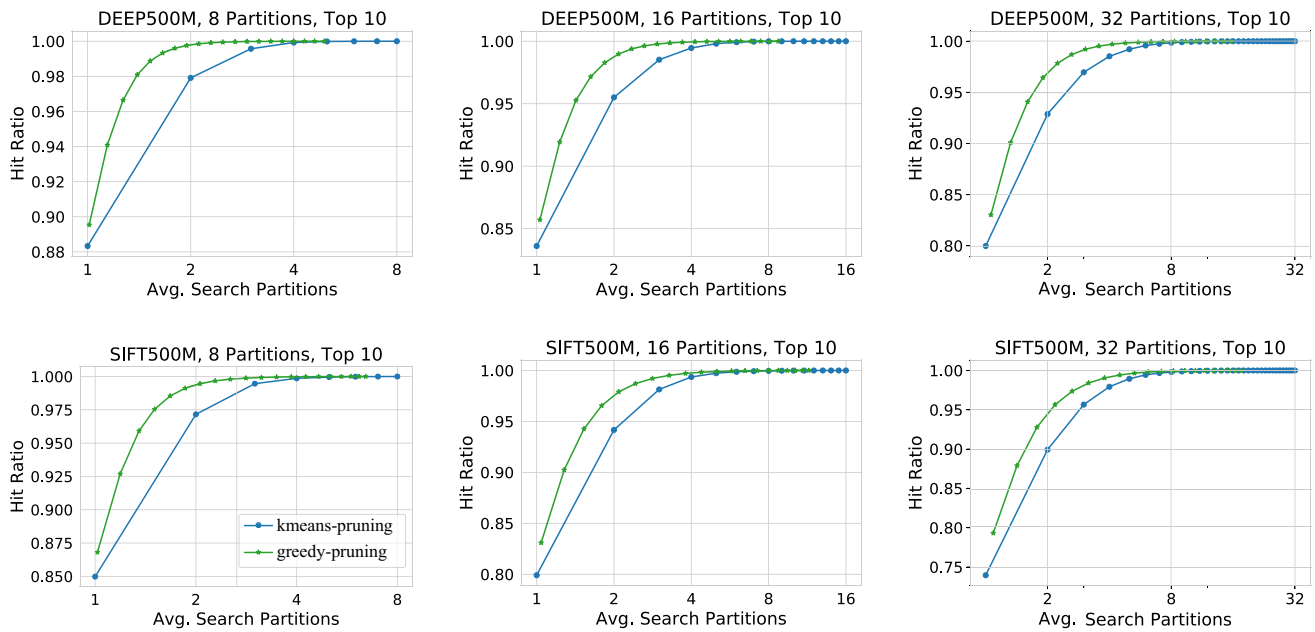


Fig. 8 Effect of partition pruning for two-level approaches

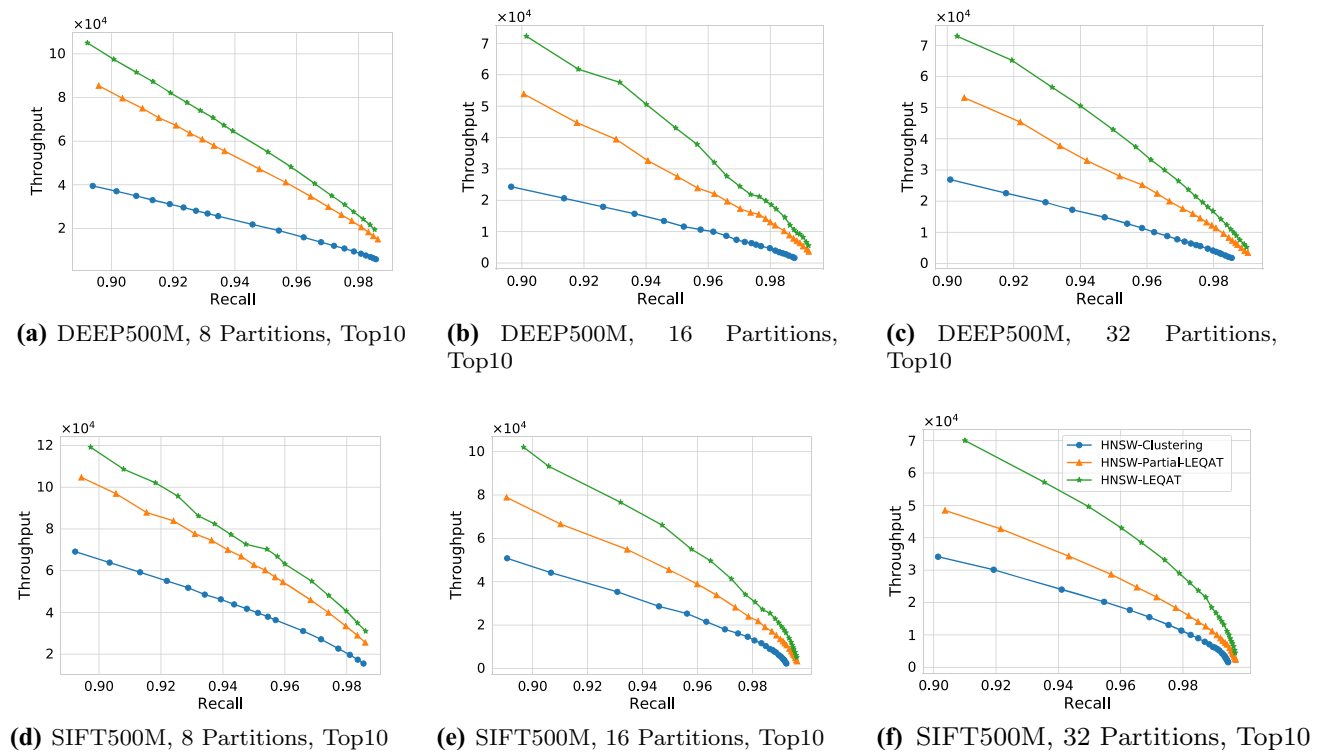


Fig. 9 End-to-End throughput

Table 5 Speedup ratio over fixed configurations (10NN, HNSW case)

Dataset	Algorithm	Recall				
		0.90	0.92	0.94	0.96	0.98
DEEP500M	LEQAT- <i>dp-pruning</i>	2.77x	2.68x	2.67x	2.89x	3.29x
	LEQAT- <i>greedy-pruning</i>	2.69x	2.61x	2.60x	2.82x	3.21x
SIFT500M	LEQAT- <i>dp-pruning</i>	2.02x	1.98x	1.78x	1.87x	2.09x
	LEQAT- <i>greedy-pruning</i>	1.82x	1.82x	1.67x	1.80x	2.05x

Each dataset is split to eight partitions through K-means clustering

Table 6 End-to-end latency of two-level approaches (ms)

		Top10 (0.9)	Top10 (0.98)	Top100 (0.9)	Top100 (0.98)
DEEP	HNSW-Clustering	0.83	3.89	1.53	8.43
	HNSW-LEQAT	0.77	3.14	1.55	6.89
	SSG-Clustering	1.52	4.85	2.62	13.16
	SSG-LEQAT	1.47	4.79	2.47	10.21
SIFT	HNSW-Clustering	0.66	2.21	1.36	5.68
	HNSW-LEQAT	0.65	2.01	1.36	5.01
	SSG-Clustering	1.22	3.31	2.04	7.40
	SSG-LEQAT	1.21	3.04	2.12	6.60

Each dataset is split to 8 partitions through kmeans clustering

then decreasing τ to acquire desirable performance. Furthermore, compared to *dp-pruning*, *greedy-pruning* achieves similar performance, with less hyperparameters, thus we use *greedy-pruning* as the default optimization algorithm in the rest experiments.

Effect of Partition Pruning Let *kmeans-pruning* denote the partition pruning approach that picks first M partitions whose centroids are closest to the query vectors to search. We evaluate the performance of *kmeans-pruning* and *greedy-pruning* in LEQAT for two-level multi-probe ANNS approaches in this section. We use hit ratio, which means the proportion of query's k NNs placed in the probed partitions, to measure the quality of partition pruning. Figure 8 shows the relation between hit ratio and average search partitions of the two evaluated partition pruning methods. Our method *greedy-pruning* outperforms *kmeans-pruning* under all settings and could reduce up to 45% search partitions under the same recall targets. The main reason is that our method considers queries' distinctness and searches different numbers of partitions for different queries.

Extensive End-to-End Evaluation We compare the end-to-end query throughput and latency of three methods: (1) *X*-Clustering (using fixed-configurations and distance-based partition pruning), (2) *X*-Partial-LEQAT (LEQAT with only *greedy-pruning* and setting c as a fixed parameter) and (3) *X*-LEQAT (LEQAT with both *greedy-pruning* and optimized configuration of c).

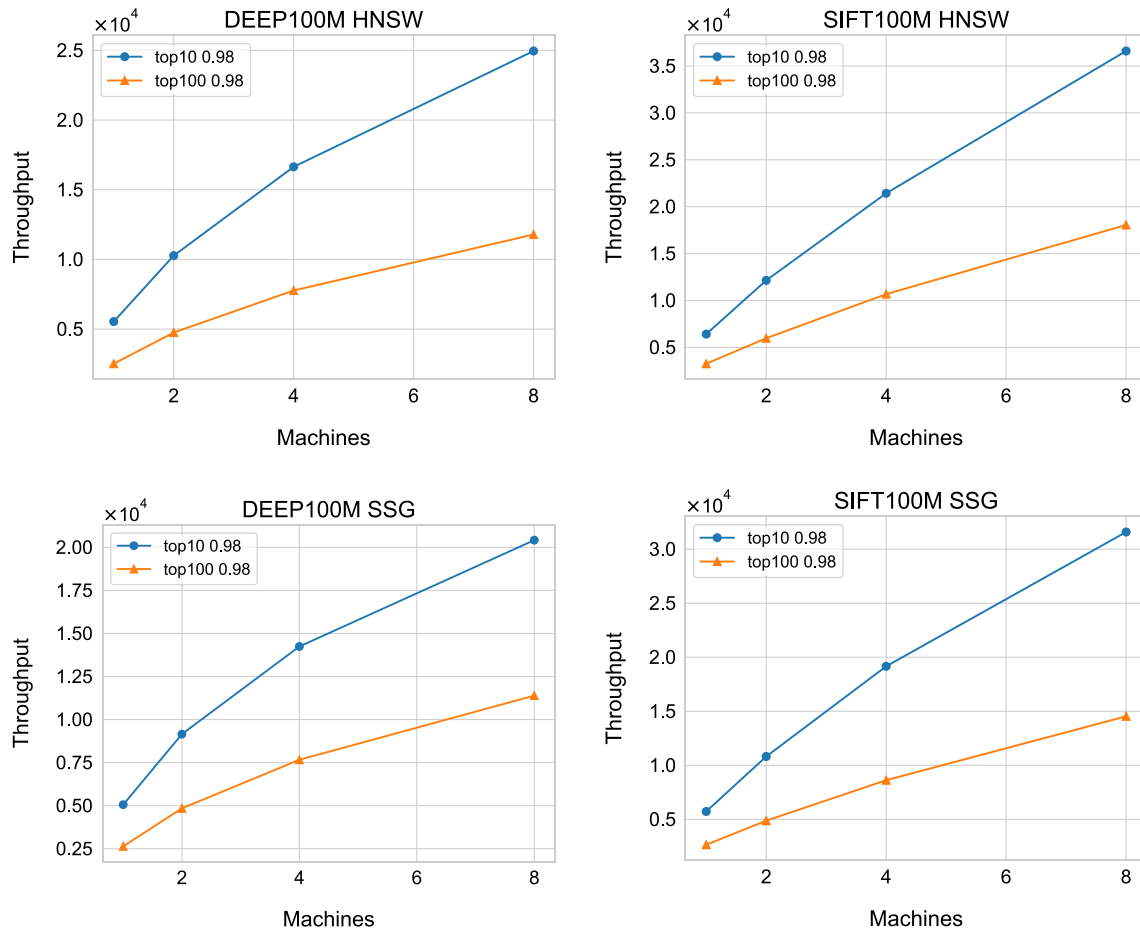
Table 6 shows the latency of the evaluated methods under the 0.9 and 0.98 recall targets and eight partitions. LEQAT achieves lower latency in most cases. Note that

LEQAT does not have a significant superiority of latency here, because the latency of distributed ANNS is majorly influenced by the partition that takes the longest time. And the throughput metric is more important in distributed settings. Figure 9 reports the query throughput under different datasets for the HNSW case. Firstly, HNSW-LEQAT outperforms the baseline method under all settings. Also, we could achieve a significant speedup with only *greedy-pruning* in LEQAT (HNSW-Partial-LEQAT). Moreover, the full version of LEQAT (HNSW-LEQAT) always outperforms the partial version. It proves the effectiveness of query-aware configurations generated by LEQAT. Table 7 summarizes the speedup of LEQAT over the baseline using HNSW and SSG indexes. LEQAT consistently improves 1.5x–3.9x query throughput under the same recall targets.

Scalability Evaluation We also test the scalability of LEQAT for two-level multi-probe ANNS. The experiment is conducted on DEEP100M and SIFT100M composed of the first 100M vectors in DEEP and SIFT. Figure 10 illustrates the top-10 and top-100 query throughput of LEQAT with HNSW and SSG under the 0.98 recall. We use 1, 2, 4, and 8 machines, and for each test we set the partition number equals to the machine number. As the number of machines doubles, the query throughput of LEQAT increased by 150%~185%. The reason why LEQAT cannot achieve exact linear scaling may be due to the graph index's logarithmic search complexity. The search complexity of a graph index is scaled by $O(\log n)$, where n is the dataset size. As the machine number doubles, the size of the subset processed by a machine halves. Due to the logarithmic search complexity, the search throughput for

Table 7 LEQAT speedup ratio over baselines (for two-level approaches)

		HNSW Case				SSG Case			
		Top10 (0.9)	Top10 (0.98)	Top100 (0.9)	Top100 (0.98)	Top10 (0.9)	Top10 (0.98)	Top100 (0.9)	Top100 (0.98)
$N = 8$	DEEP	2.69	3.21	2.61	3.34	2.29	2.76	2.40	3.43
	SIFT	1.82	2.05	1.56	2.06	1.53	1.73	1.53	1.79
$N = 16$	DEEP	2.88	3.96	2.68	3.94	2.45	3.47	2.32	3.37
	SIFT	1.90	2.10	1.89	2.46	1.70	1.90	1.59	2.08
$N = 32$	DEEP	2.24	3.74	2.25	3.90	2.08	3.28	2.11	3.51
	SIFT	1.87	2.27	1.78	2.35	1.80	2.06	1.76	2.16

**Fig. 10** Scalability

a subset will not double. Thus LEQAT cannot achieve exact linear scaling. However, since LEQAT only probes a small proportion of partitions for each query, it could also achieve considerable sub-linear scalability.

7 Related work

Single-machine ANNS Approaches Many single-machine approaches have been proposed to solve the ANNS prob-

lem, including tree-based methods [6,21,40,43], hash-based methods [11,18,24,48], and quantization-based methods [17,19,27,30,52]. Some well-known and widely used approaches, such as Locality Sensitive Hashing (LSH) [18] and Product Quantization (PQ) [27], all fall into the above categories. Recently, graph-based ANNS approaches have demonstrated outstanding accuracy and efficiency trade-off [14,15,35,37,38]. These methods build proximity graphs to index the database vectors. ANNS is performed by traversing the proximity graph.

Hierarchical Navigable Small World Graph (HNSW) is one of the state-of-the-art graph-based ANNS method, which achieves high search performance through the hierarchical graph structure and small-world graph properties [38]. Later, Fu, et al. proposed Navigating Spreading-out Graph (NSG), which achieves similar performance as HNSW but with theoretical proof of the logarithmic search complexity [15]. Satellite System Graph (SSG) is an improvement of NSG that achieves a better accuracy–efficiency trade-off and has lower index building cost [14] by optimizing the graph connections. HNSW, NSG, and SSG all save the index structure and database vectors in the memory, making them hard to scale to large datasets on a single machine.

Distributed ANNS Approaches To process large datasets, distributed ANNS solutions are proposed, including tree-based methods [39], LSH-based methods [49,59], and methods incorporating proximity graphs [10,15,53]. In particular, NSG [15] randomly splits a dataset evenly into multiple disjoint subsets and builds a NSG index for each of them. For top- k ANNS, this approach searches top- c candidate neighbors in all partitions. c is a fixed parameter for all queries to control the accuracy–efficiency trade-off. ADBV [53] further explores the partitioning technique based on K-means [22] clustering, which can improve its query throughput by 10x to 100x. In each partition, it builds an in-memory HNSW index to save and query recent inserted or modified data and periodically merge the in-memory index to the on-disk index based on quantization. ADBV’s partition pruning procedure and search on the in-memory HNSW index are also based on a fixed-configuration manner.

Machine Learning and NNS Recent work by Kraska et al. [32] proposed the idea of learned index, which replaces traditional database indexes such as B-tree with learned models for one-dimensional data. Given a search key, the learned index predicts the position of the corresponding data record and guarantees that the record is found around the predicted position within a known error bound. Several studies [34,50] extended the idea to multi-dimensional data and proposed learned indexes supporting exact NNS. There are also studies on optimization of NNS variants based on machine learning [2,42,47,51], e.g., learning-based cardinality estimation [2,47,51] for similarity search under a specific distance limit.

As to ANNS, Dong et al. [12] proposed the learned space partitioning approach that aims to generate high-quality partitions to serve ANNS. This approach builds a k NN graph for a high-dimensional vector dataset and splits the graph into several bins. Then, a machine learning model is trained that given any vector, it will predict the several possible bins that the vector may belong to. This work is focused on space partitioning, and ANNS is also performed in a fixed-configuration manner. The recent study by Li et al. [33] identified inefficiency of existing ANNS methods such as IVF [29], IMI [4], and HNSW [38] due to using fixed configurations. The

authors then proposed a learning model to predict the termination condition, i.e., the maximum number of distance computations to find the exact top-1 nearest neighbor for each query individually. The top-1 ANNS is performed by a two-step algorithm. Firstly the algorithm searches the index by a fixed amount of distance computations to collect intermediate features. Then the collected features are fed to the learning model to predict the termination condition, and the search is continued until the termination condition is met. Our study distinguishes this work from several aspects. Firstly, this work focuses on optimizing top-1 ANNS based on the prediction of the termination condition. Our work is about top- k ANNS based on our novel k NN distribution estimation technique. Secondly, LEQAT is a one-pass optimizer that generates efficient configurations for existing algorithms. However, the approach [33] works in a two-round manner and needs to modify existing search algorithms to apply the two-round approach.

8 Conclusion

State-of-the-art multi-probe ANNS solutions generally split the original dataset into several partitions and use fixed configurations to balance accuracy and efficiency. However, our empirical results indicate that such fixed configurations lead to a non-optimal accuracy–efficiency trade-off, because they fail to consider the diversity of queries and partitions. In this paper, we have demonstrated that we could optimize multi-probe ANNS performance by exploiting the k NN distribution of different queries. We have presented the novel techniques, including k NN distribution estimation and query-aware search configuration, to determine efficient configurations for queries individually. Our proposed approaches substantially improve the SOTA methods on the end-to-end performance.

Acknowledgements This work was supported by the NSFC (61922054, 61872235, 61832017, 61729202, 61832013), the National Key Research and Development Program of China (2020YFB1710200), the Science and Technology Commission of Shanghai Municipality (STCSM) AI under Project 19511120300 and Hangzhou Qianjiang Distinguished Expert Program.

References

- Adeniyi, D.A., Wei, Z., Yongquan, Y.: Automated web usage data mining and recommendation system using k -nearest neighbor (k NN) classification method. *Appl. Comput. Inform.* **12**, 90–108 (2016)
- Anagnostopoulos, C., Triantafillou, P.: Learning set cardinality in distance nearest neighbours. In: *ICDM*, pp. 691–696. IEEE (2015)
- Andoni, A., Indyk P., Laarhoven T., Razenshteyn I., Schmidt, L.: Practical and optimal LSH for angular distance. *arXiv preprint arXiv:1509.02897* (2015)

4. Babenko, A., Lempitsky, V.: The inverted multi-index. *IEEE Trans. Pattern Anal. Mach. Intell.* **37**(6), 1247–1260 (2014)
5. Babenko, A., Lempitsky, V.: Efficient indexing of billion-scale datasets of deep descriptors. In: *IEEE Conference on Computer Vision and Pattern Recognition*, pp. 2055–2063 (2016)
6. Bentley, J.L.: Multidimensional binary search trees used for associative searching. *Commun. ACM* **18**, 509–517 (1975)
7. Bijalwan, V., Kumar, V., Kumari, P., Pascual, J.: *k*NN based machine learning approach for text and document mining. *Int. J. Database Theory Appl.* **7**, 61–70 (2014)
8. Chauvin, Y., Rumelhart, D.E.: *Backpropagation: Theory, Architectures, and Applications*. Psychology Press, London (1995)
9. Chen, L., Özsü, M.T., Oria, V.: Robust and fast similarity search for moving object trajectories. In: *SIGMOD*, pp. 491–502 (2005)
10. Chen, Q., Wang, H., Li, M., Ren, G., Li, S., Zhu, J., Li, J., Liu, C., Zhang, L., Wang, J.: SPTAG: a library for fast approximate nearest neighbor search (2018)
11. Datar, M., Immorlica, N., Indyk, P., Mirrokni, V.S.: Locality-sensitive hashing scheme based on *p*-stable distributions. In: *Symposium on Computational Geometry*, pp. 253–262 (2004)
12. Dong, Y., Indyk, P., Razenshteyn, I., Wagner, T.: Learning space partitions for nearest neighbor search. In: *ICLR* (2020)
13. Dudziński, K., Walukiewicz, S.: Exact methods for the knapsack problem and its generalizations. *Eur. J. Oper. Res.* **28**(1), 3–21 (1987)
14. Fu, C., Wang, C., Cai, D.: Satellite system graph: towards the efficiency up-boundary of graph-based approximate nearest neighbor search. *CoRR*, [arXiv:1907.06146](https://arxiv.org/abs/1907.06146) (2019)
15. Fu, C., Xiang, C., Wang, C., Cai, D.: Fast approximate nearest neighbor search with the navigating spreading-out graph. *VLDB* **12**, 461–474 (2019)
16. Gardner, M.W., Dorling, S.: Artificial neural networks (the multilayer perceptron)-a review of applications in the atmospheric sciences. *Atmos. Environ.* **32**(14–15), 2627–2636 (1998)
17. Ge, T., He, K., Ke, Q., Sun, J.: Optimized product quantization for approximate nearest neighbor search. In: *IEEE Conference on Computer Vision and Pattern Recognition*, pp. 2946–2953 (2013)
18. Gionis, A., Indyk, P., Motwani, R., et al.: Similarity search in high dimensions via hashing. *VLDB* **99**, 518–529 (1999)
19. Gray, R.: Vector quantization. *IEEE Assp Mag.* **1**, 4–29 (1984)
20. Guo, G., Wang, H., Bell, D., Bi, Y., Greer, K.: *k*NN model-based approach in classification. In: *OTM Confederated International Conferences “On the Move to Meaningful Internet Systems”*, pp. 986–996 (2003)
21. Guttman, A.: R-trees: a dynamic index structure for spatial searching. In: *ACM* (1984)
22. Hartigan, J.A., Wong, M.A.: Algorithm as 136: a *k*-means clustering algorithm. *J. R. Stat. Soc. Ser. C (Appl. Stat.)* **28**, 100–108 (1979)
23. Hilbe, J.M.: *Logistic Regression Models*. Chapman and Hall/CRC, Boca Raton (2009)
24. Huang, Q., Feng, J., Zhang, Y., Fang, Q., Ng, W.: Query-aware locality-sensitive hashing for approximate nearest neighbor search. *VLDB* **9**, 1–12 (2015)
25. Indyk, P., Motwani, R.: Approximate nearest neighbors: towards removing the curse of dimensionality. In: *ACM Symposium on Theory of Computing*, pp. 604–613 (1998)
26. Jegou, H., Douze, M., Schmid, C.: Product quantization for nearest neighbor search. *IEEE Trans. Pattern Anal. Mach. Intell.* **33**(1), 117–128 (2010)
27. Jégou, H., Douze, M., Schmid, C.: Product quantization for nearest neighbor search. *IEEE Trans. Pattern Anal. Mach. Intell.* **33**, 117–128 (2011)
28. Jégou, H., Tavenard, R., Douze, M., Amsaleg, L.: Searching in one billion vectors: re-rank with source coding. In: *ICASSP*, pp. 861–864 (2011)
29. Johnson, J., Douze, M., Jégou, H.: Billion-scale similarity search with gpus. *IEEE Trans. Big Data* **7**(3), 535–547 (2019)
30. Kalantidis, Y., Avrithis, Y.: Locally optimized product quantization for approximate nearest neighbor search. In: *IEEE Conference on Computer Vision and Pattern Recognition*, pp. 2321–2328 (2014)
31. Kellerer, H., Pferschy, U., Pisinger, D.: The multiple-choice knapsack problem. In: *Knapsack Problems*, pp. 317–347. Springer, Berlin, Heidelberg (2004). https://doi.org/10.1007/978-3-540-24777-7_11
32. Kraska, T., Beutel, A., Chi, E.H., Dean, J., Polyzotis, N.: The case for learned index structures. In: *SIGMOD*, pp. 489–504 (2018)
33. Li, C., Zhang, M., Andersen, D.G., He, Y.: Improving approximate nearest neighbor search through learned adaptive early termination. In: *SIGMOD*, pp. 2539–2554 (2020)
34. Li, P., Lu, H., Zheng, Q., Yang, L., Pan, G.: Lisa: a learned index structure for spatial data. In: *SIGMOD*, pp. 2119–2133 (2020)
35. Li, W., Zhang, Y., Sun, Y., Wang, W., Li, M., Zhang, W., Lin, X.: Approximate nearest neighbor search on high dimensional data-experiments, analyses, and improvement. *IEEE Trans. Knowl. Data Eng.* **32**, 1475–1488 (2019)
36. Lv, Q., Josephson, W., Wang, Z., Charikar, M., Li, K.: Multi-probe LSH: efficient indexing for high-dimensional similarity search. In: *33rd International Conference on Very Large Data Bases, VLDB 2007*, pp. 950–961. Association for Computing Machinery, Inc. (2007)
37. Malkov, Y., Ponomarenko, A., Logvinov, A., Krylov, V.: Approximate nearest neighbor algorithm based on navigable small world graphs. *Inf. Syst.* **45**, 61–68 (2014)
38. Malkov, Y.A., Yashunin, D.A.: Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* **42**, 824–836 (2018)
39. Muja, M., Lowe, D.G.: Scalable nearest neighbor algorithms for high dimensional data. *IEEE Trans. Pattern Anal. Mach. Intell.* **36**, 2227–2240 (2014)
40. Omohundro, S.M.: Five balltree construction algorithms. In: *International Computer Science Institute Berkeley* (1989)
41. Pennington, J., Socher, R., Manning, C.D.: Glove: global vectors for word representation. In: *EMNLP*, pp. 1532–1543 (2014)
42. Qin, J., Wang, W., Xiao, C., Zhang, Y.: Similarity query processing for high-dimensional data. *VLDB* **13**(12), 3437–3440 (2020)
43. Robinson, J.T.: The KDB-tree: a search structure for large multi-dimensional dynamic indexes. In: *SIGMOD*, pp. 10–18 (1981)
44. Rui, Y., Huang, T.S., Chang, S.-F.: Image retrieval: current techniques, promising directions, and open issues. *J. Vis. Commun. Image Represent.* **10**, 39–62 (1999)
45. Shamos, M.I., Hoey, D.: Closest-point problems. In: *Symposium on Foundations of Computer Science*, pp. 151–162 (1975)
46. Suchal, J., Návrát, P.: Full text search engine as scalable *k*-nearest neighbor recommendation system. In: *International Conference on Artificial Intelligence in Theory and Practice*, pp. 165–173 (2010)
47. Sun, J., Li, G., Tang, N.: Learned cardinality estimation for similarity queries. In: *SIGMOD*, pp. 1745–1757 (2021)
48. Sun, Y., Wang, W., Qin, J., Zhang, Y., Lin, X.: SRS: solving *c*-approximate nearest neighbor queries in high dimensional Euclidean space with a tiny index. *VLDB* **8**, 1–12 (2014)
49. Sundaram, N., Turmukhametova, A., Satish, N., Mostak, T., Indyk, P., Madden, S., Dubey, P.: Streaming similarity search over one billion tweets using parallel locality-sensitive hashing. *VLDB* **6**, 1930–1941 (2013)
50. Wang, H., Fu, X., Xu, J., Lu, H.: Learned index for spatial queries. In: *MDM*, pp. 569–574. *IEEE* (2019)
51. Wang, Y., Xiao, C., Qin, J., Cao, X., Sun, Y., Wang, W., Onizuka, M.: Monotonic cardinality estimation of similarity selection: a deep learning approach. In: *SIGMOD*, pp. 1197–1212 (2020)

52. Weber, R., Schek, H.-J., Blott, S.: A quantitative analysis and performance study for similarity-search methods in high-dimensional spaces. *VLDB* **98**, 194–205 (1998)
53. Wei, C., Wu, B., Wang, S., Lou, R., Zhan, C., Li, F., Cai, Y.: Analyticdb-v: a hybrid analytical engine towards query fusion for structured and unstructured data. *VLDB* **13**, 3152–3165 (2020)
54. Xie, D., Li, F., Yao, B., Li, G., Zhou, L., Guo, M.: Simba: efficient in-memory spatial analytics. In: *SIGMOD*, pp. 1071–1085 (2016)
55. Yang, W., Li, T., Fang, G., Wei, H.: Pase: postgresql ultra-high-dimensional approximate nearest neighbor search extension. In: *SIGMOD*, pp. 2241–2253 (2020)
56. Yao, B., Li, F., Kumar, P.: K nearest neighbor queries and k NN-joins in large relational databases (almost) for free. In: *IEEE International Conference on Data Engineering (ICDE)*, pp. 4–15 (2010)
57. Yu, J., Wu, J., Sarwat, M.: Geospark: a cluster computing framework for processing large-scale spatial data. In: *ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*, pp. 1–4 (2015)
58. Zhang, S., Li, X., Zong, M., Zhu, X., Cheng, D.: Learning k for k NN classification. *ACM Trans. Intell. Syst. Technol. (TIST)* **8**, 1–19 (2017)
59. Zhang, W., Li, D., Xu, Y., Zhang, Y.: Shuffle-efficient distributed locality sensitive hashing on spark. In: *IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, pp. 766–767 (2016)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.